

Lecture 2:

Pipelining and Instruction-Level Parallelism

15-418 Parallel Computer Architecture and Programming
CMU 15-418/15-618, Spring 2019

Many kinds of processors



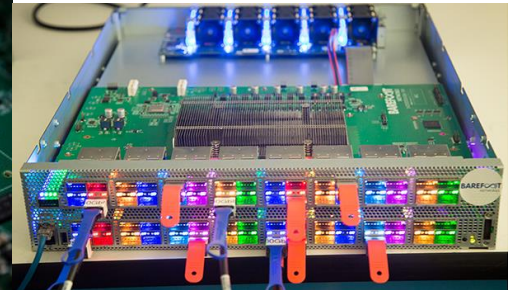
CPU



GPU



FPGA



Etc.

Why so many? What differentiates these processors?

Why so many kinds of processors?

Each processor is designed for different kinds of programs

- CPUs

- “Sequential” code – i.e., single / few threads

- GPUs

- Programs with lots of independent work → “Embarrassingly parallel”

- Many others: Deep neural networks, Digital signal processing, Etc.

Parallelism pervades architecture

- Speeding up programs is all about parallelism
 - 1) Find independent work
 - 2) Execute it in parallel
 - 3) Profit
- Key questions:
 - Where is the parallelism?
 - Whose job is it to find parallelism?

Where is the parallelism?

Different processors take radically different approaches

- CPUs: Instruction-level parallelism

ILP

- Implicit
- Fine-grain

- GPUs: Thread- & data-level parallelism

- Explicit
- Coarse-grain

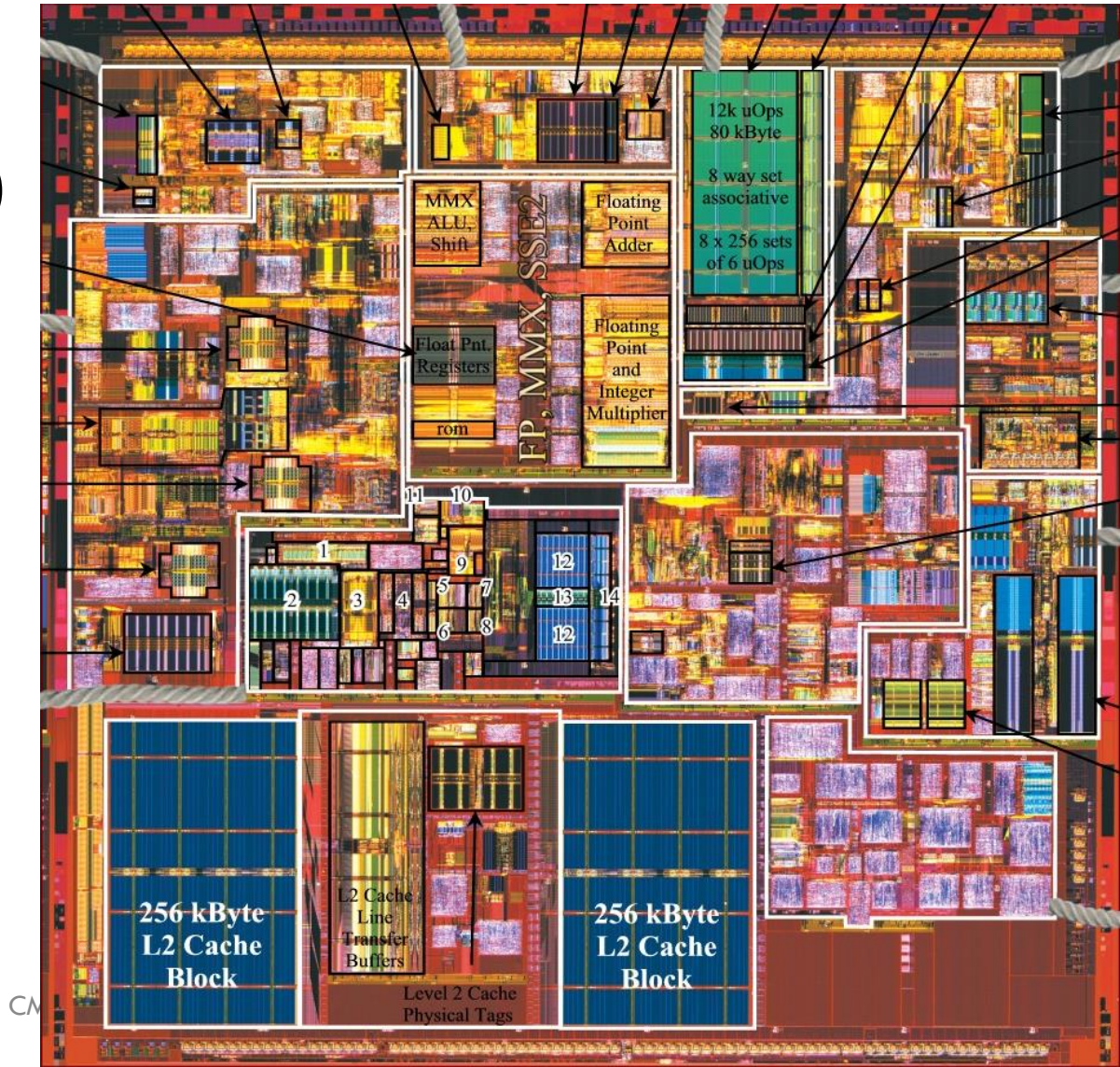
Whose job to find parallelism?

Different processors take radically different approaches

- CPUs: Hardware dynamically schedules instructions
 - Expensive, complex hardware → Few cores (tens)
 - (Relatively) Easy to write fast software
- GPUs: Software makes parallelism explicit
 - Simple, cheap hardware → Many cores (thousands)
 - (Often) Hard to write fast software

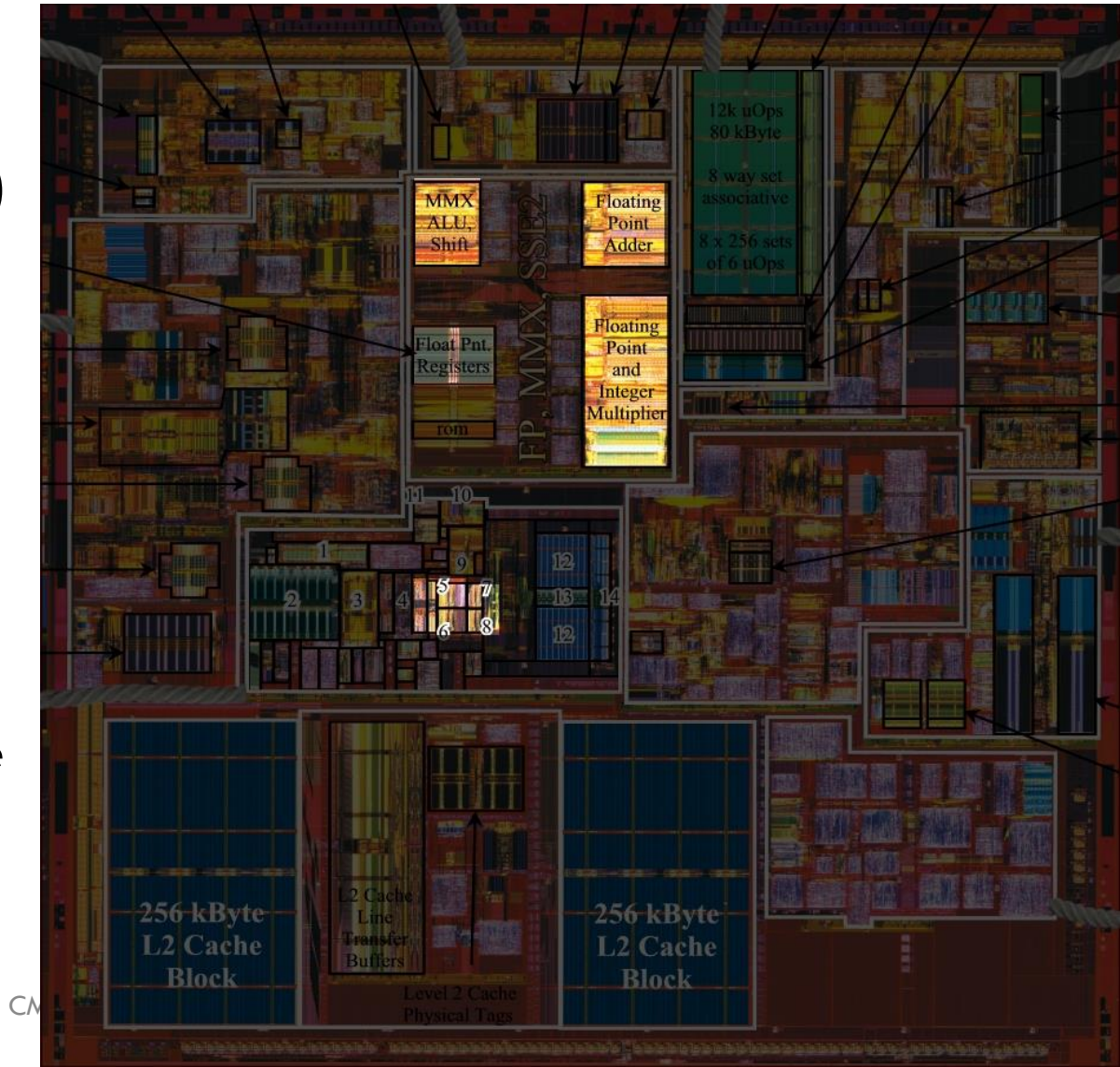
Visualizing these differences

- Pentium 4
“Northwood” (2002)



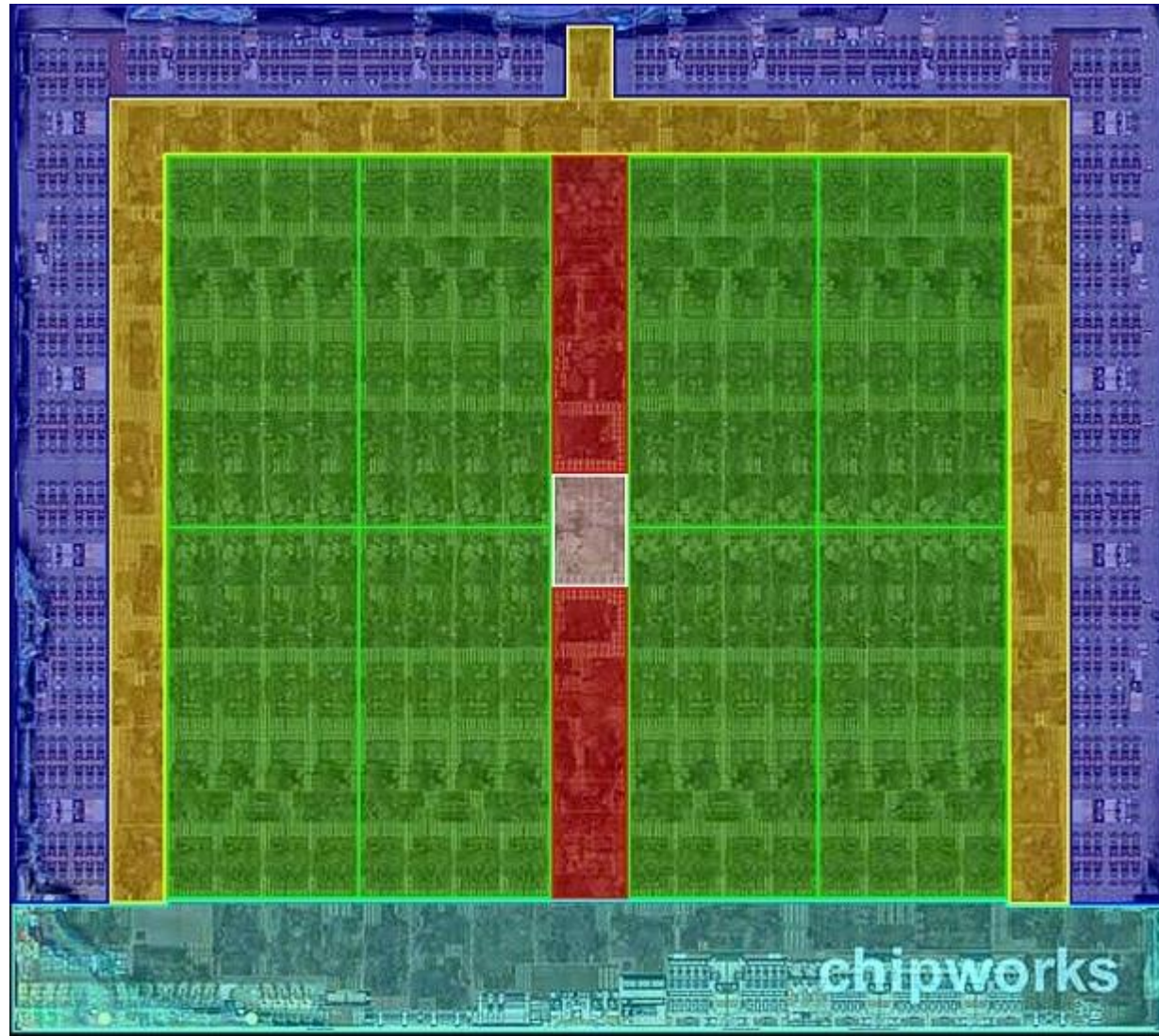
Visualizing these differences

- Pentium 4
“Northwood” (2002)
- Highlighted areas actually execute instructions
 - ➔ Most area spent on scheduling
(not on executing the program)



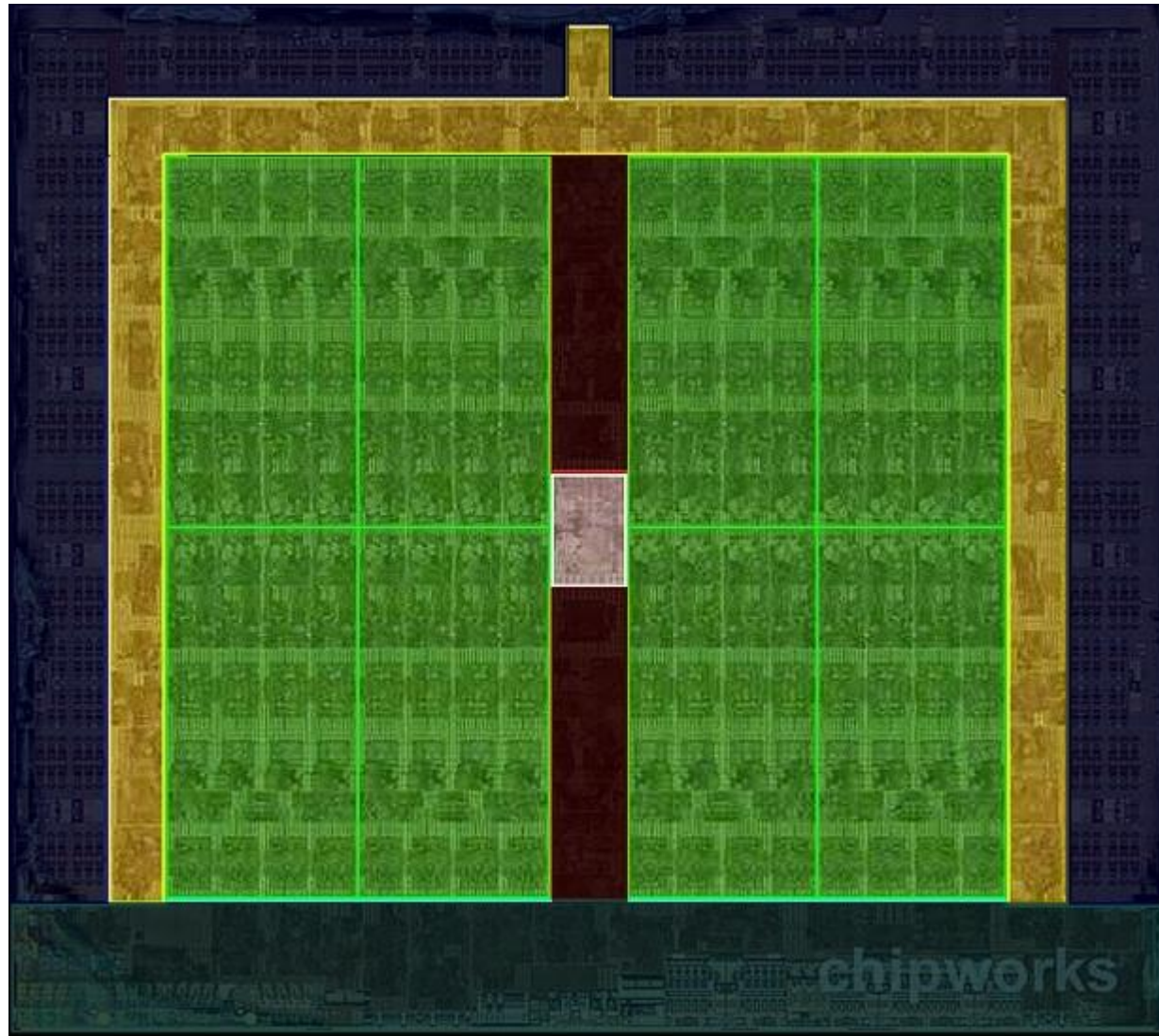
Visualizing these differences

- AMD Fiji (2015)



Visualizing these differences

- AMD Fiji (2015)
- Highlighted areas actually execute instructions
 - ➔ Most area spent executing the program
 - (Rest is mostly I/O & memory, not scheduling)

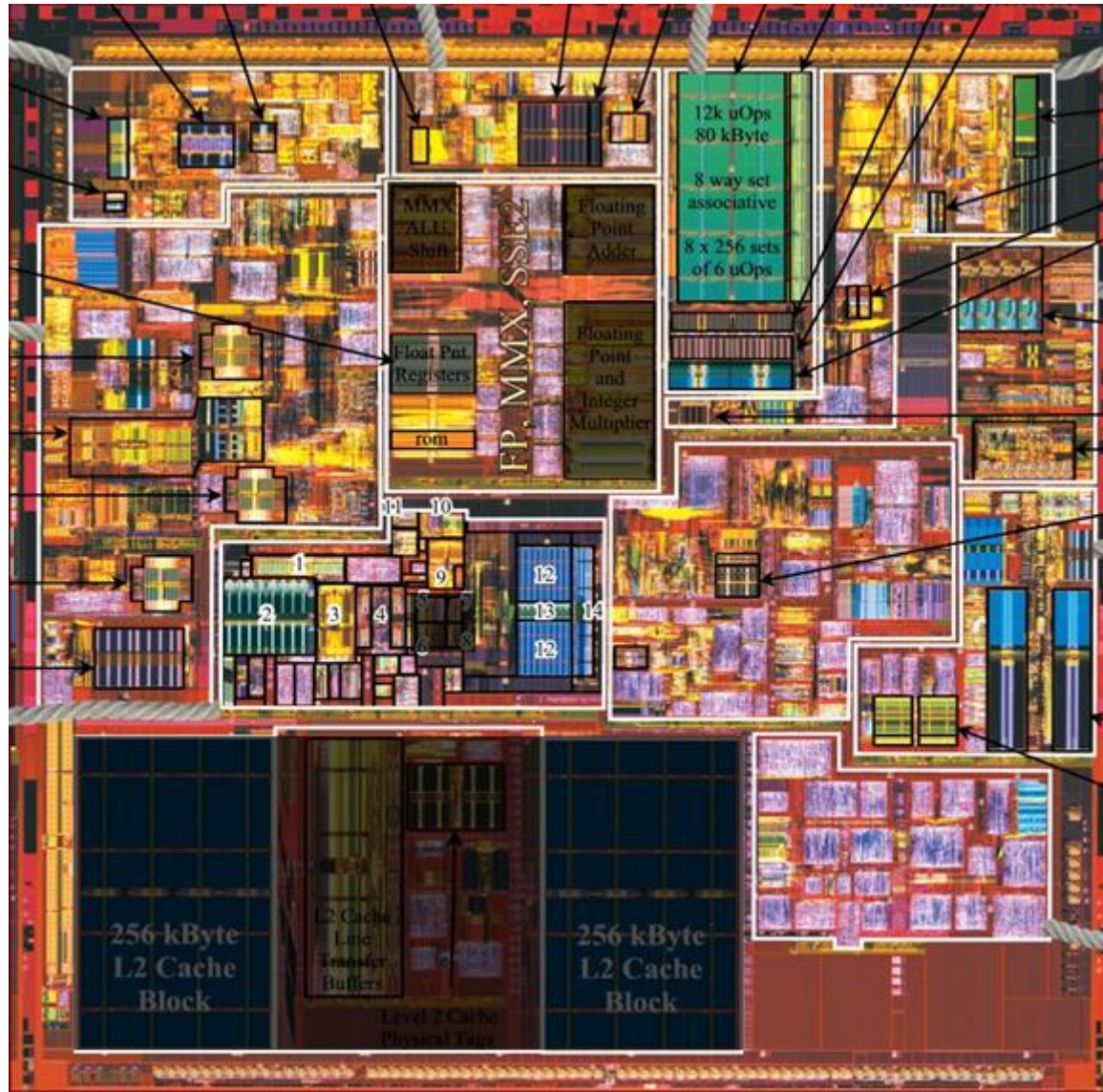


Today you will learn...

How CPUs exploit ILP to speed up straight-line code

- Key ideas:
 - Pipelining & Superscalar: Work on multiple instructions at once
 - Out-of-order execution: Dynamically schedule instructions whenever they are “ready”
 - Speculation: Guess what the program will do next to discover more independent work, “rolling back” incorrect guesses
- CPUs must do all of this while preserving the illusion that instructions execute in-order, one-at-a-time

In other words... Today is about:



Buckle up!

...But please ask questions!

Example:

Polynomial evaluation

```
int poly(int *coef,  
         int terms, int x) {  
    int power = 1;  
    int value = 0;  
    for (int j = 0; j < terms; j++) {  
        value += coef[j] * power;  
        power *= x;  
    }  
    return value;  
}
```


r0: value
r1: &coef[terms]
r2: x
r3: &coef[0]
r4: power
r5: coef[j]

Example:

Polynomial evaluation

■ Compiling on ARM

```
int poly(int *coef,  
        int terms, int x) {  
    int power = 1;  
    int value = 0;  
    for (int j = 0; j < terms; j++) {  
        value += coef[j] * power;  
        power *= x;  
    }  
    return value;  
}
```

```
poly:  
    cmp     r1, #0  
    ble     .L4  
    push    {r4, r5}  
    mov     r3, r0  
    add     r1, r0, r1, lsl #2  
    movs    r4, #1  
    movs    r0, #0  
    .L3:  
    ldr     r5, [r3], #4  
    cmp     r1, r3  
    mla     r0, r4, r5, r0  
    mul     r4, r2, r4  
    bne     .L3  
    pop     {r4, r5}  
    bx      lr  
    .L4:  
    movs    r0, #0  
    bx      lr
```

Example:

Polynomial evaluation

r0: value
 r1: &coef[terms]
 r2: x
 r3: &coef[0]
 r4: power
 r5: coef[j]

■ Compiling on ARM

```

int poly(int *coef,
        int terms, int x) {
    int power = 1;
    int value = 0;

    for (int j = 0; j < terms; j++) {
        value += coef[j] * power;
        power *= x;
    }

    return value;
}
  
```

poly:	cmp	r1, #0	Preamble
	ble	.L4	
	push	{r4, r5}	
	mov	r3, r0	
	add	r1, r0, r1, lsl #2	
	movs	r4, #1	
	movs	r0, #0	
.L3:	ldr	r5, [r3], #4	Iteration
	cmp	r1, r3	
	mla	r0, r4, r5, r0	
	mul	r4, r2, r4	
	bne	.L3	
	pop	{r4, r5}	Fini
	bx	lr	
.L4:	movs	r0, #0	
	bx	lr	

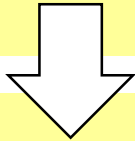
r0: value
r1: &coef[terms]
r2: x
r3: &coef[j]
r4: power
r5: coef[j]

Example:

Polynomial evaluation

■ Compiling on ARM

```
for (int j = 0; j < terms; j++) {  
    value += coef[j] * power;  
    power *= x;  
}
```



```
.L3:  
    ldr    r5, [r3], #4  
    cmp    r1, r3  
    mla    r0, r4, r5, r0  
    mul    r4, r2, r4  
    bne    .L3
```

```
// r5 <- coef[j]; j++    (two operations)  
// compare: j < terms?  
// value += r5 * power    (mul + add)  
// power *= x  
// repeat?
```

Example:

Polynomial evaluation

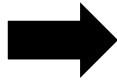
- Executing `poly(A, 3, x)`

```
cmp    r1, #0
ble    .L4
push   {r4, r5}
mov    r3, r0
add    r1, r0, r1, lsl #2
movs   r4, #1
movs   r0, #0
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
...
```

Example:

Polynomial evaluation

- Executing `poly(A, 3, x)`



```
cmp    r1, #0
ble    .L4
push   {r4, r5}
mov    r3, r0
add    r1, r0, r1, lsl #2
movs   r4, #1
movs   r0, #0
```

Preamble

```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
```

J=0 iteration

...

Example:

Polynomial evaluation

- Executing `poly(A, 3, x)`

<code>cmp</code>	<code>r1, #0</code>	Preamble
<code>ble</code>	<code>.L4</code>	
<code>push</code>	<code>{r4, r5}</code>	
<code>mov</code>	<code>r3, r0</code>	
<code>add</code>	<code>r1, r0, r1, lsl #2</code>	
<code>movs</code>	<code>r4, #1</code>	
<code>movs</code>	<code>r0, #0</code>	

<code>ldr</code>	<code>r5, [r3], #4</code>	J=0 iteration
<code>cmp</code>	<code>r1, r3</code>	
<code>mla</code>	<code>r0, r4, r5, r0</code>	
<code>mul</code>	<code>r4, r2, r4</code>	
<code>bne</code>	<code>.L3</code>	

...

...

<code>ldr</code>	<code>r5, [r3], #4</code>	J=1 iteration
<code>cmp</code>	<code>r1, r3</code>	
<code>mla</code>	<code>r0, r4, r5, r0</code>	
<code>mul</code>	<code>r4, r2, r4</code>	
<code>bne</code>	<code>.L3</code>	

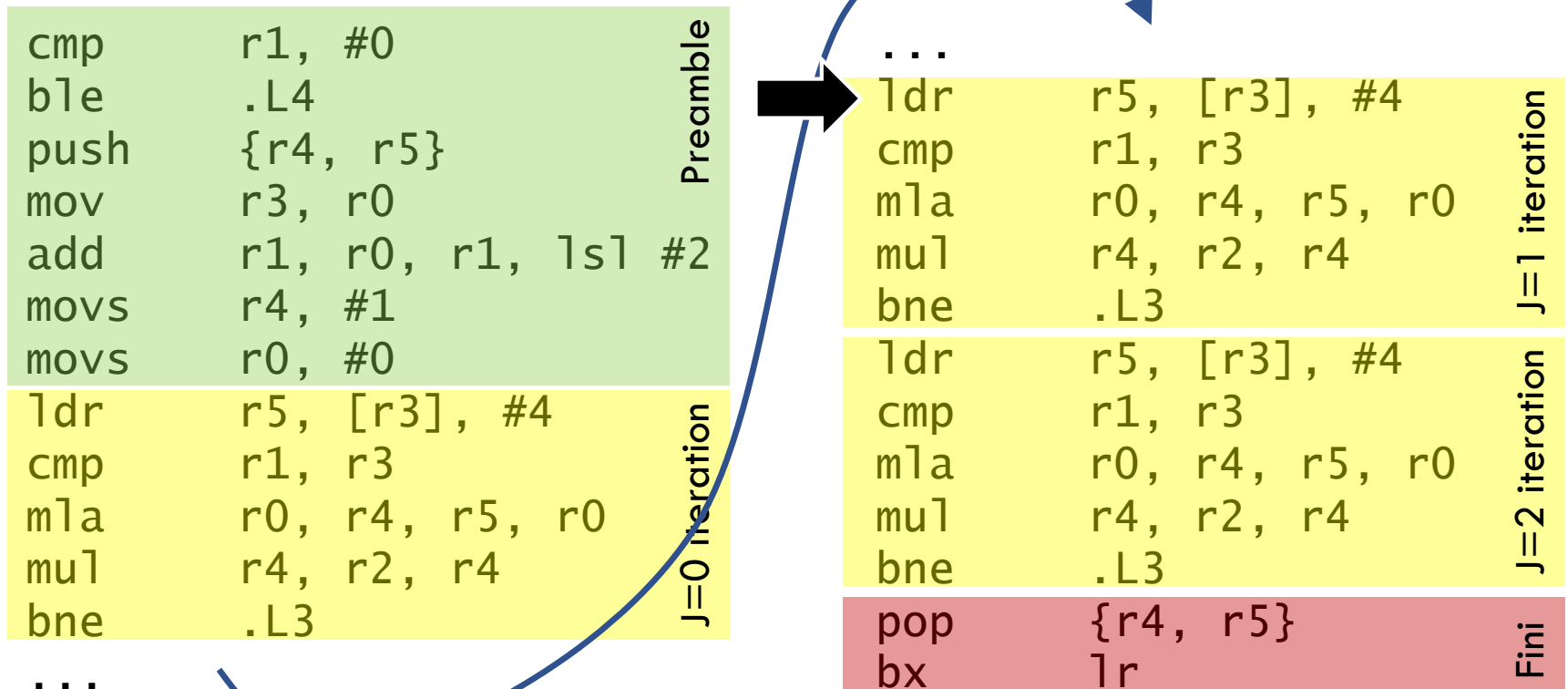
<code>ldr</code>	<code>r5, [r3], #4</code>	J=2 iteration
<code>cmp</code>	<code>r1, r3</code>	
<code>mla</code>	<code>r0, r4, r5, r0</code>	
<code>mul</code>	<code>r4, r2, r4</code>	
<code>bne</code>	<code>.L3</code>	

<code>pop</code>	<code>{r4, r5}</code>	Fin
<code>bx</code>	<code>lr</code>	

Example:

Polynomial evaluation

- Executing `poly(A, 3, x)`



The software-hardware boundary

- The *instruction set architecture* (ISA) is a functional contract between hardware and software
 - It says **what** each instruction does, but not **how**
 - Example: Ordered sequence of x86 instructions
- A processor's *microarchitecture* is how the ISA is implemented

Arch : μ Arch :: Interface : Implementation

Simple CPU model

- Execute instructions in program order
- Divide instruction execution into stages, e.g.:
 - 1. Fetch – get the next instruction from memory
 - 2. Decode – figure out what to do & read inputs
 - 3. Execute – perform the necessary operations
 - 4. Commit – write the results back to registers / memory
- (Real processors have many more stages)

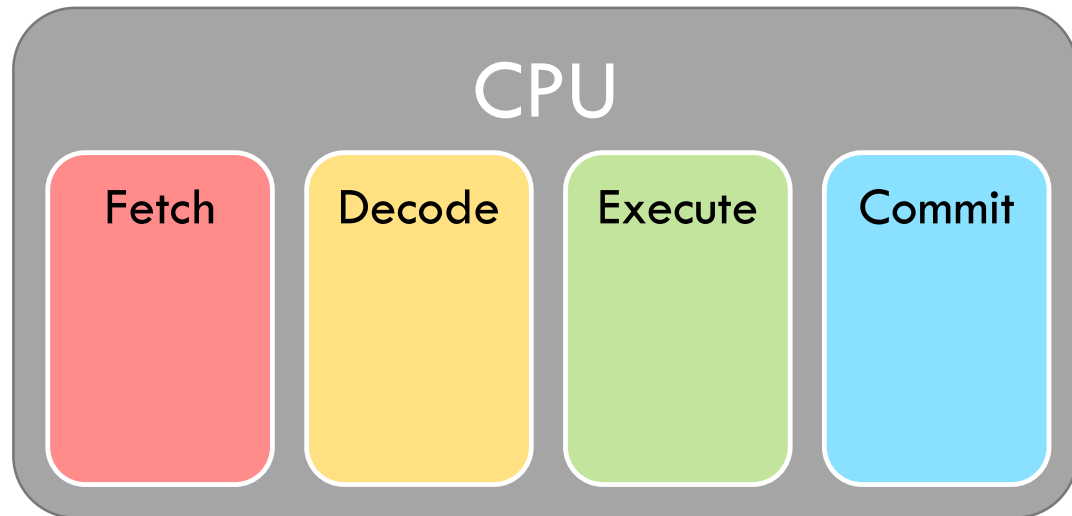
Evaluating polynomial on the simple CPU model

➔

```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3

ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3

...
```

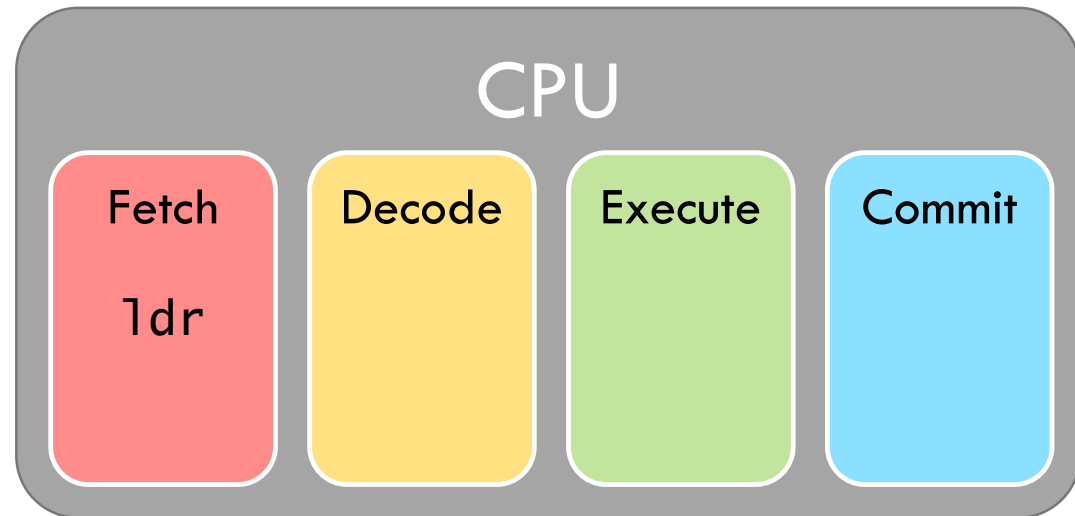


Evaluating polynomial on the simple CPU model

➔
ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

...



**1. Read "ldr r5, [r3] #4"
from memory**

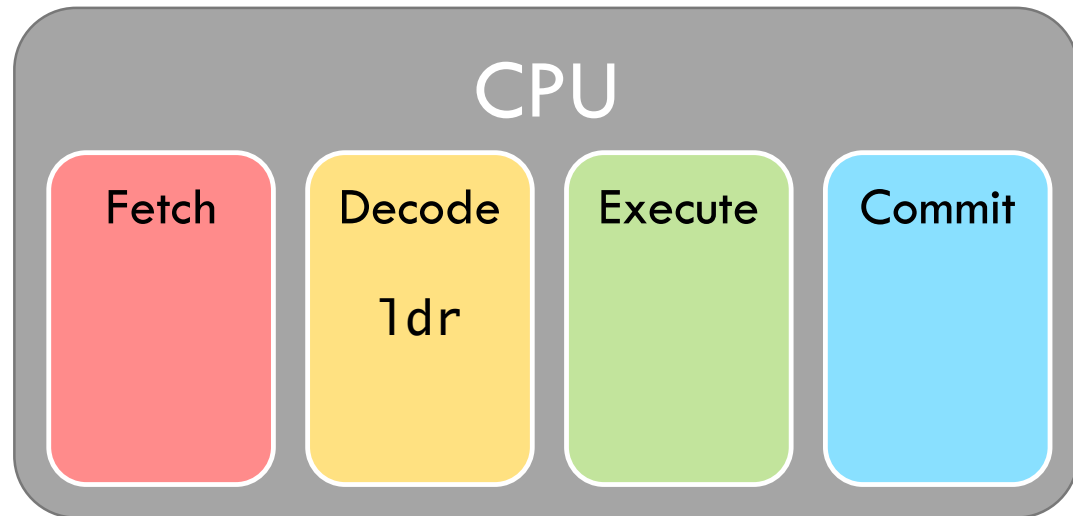
Evaluating polynomial on the simple CPU model

➔

```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3

ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3

...
```



**2. Decode "ldr r5, [r3] #4"
and read input regs**

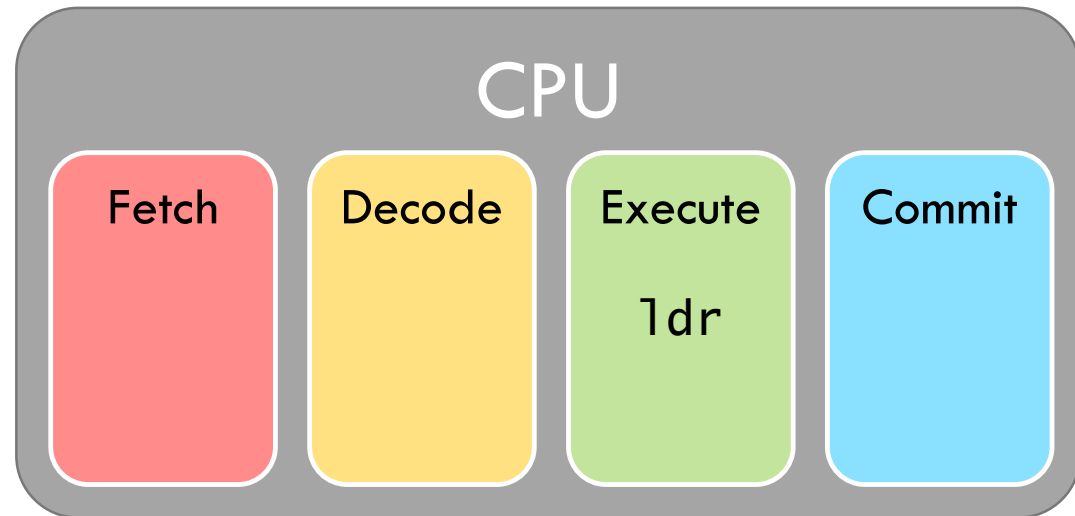
Evaluating polynomial on the simple CPU model

➔

```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3

ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3

...
```



**3. Load memory at r3 and
compute r3 + 4**

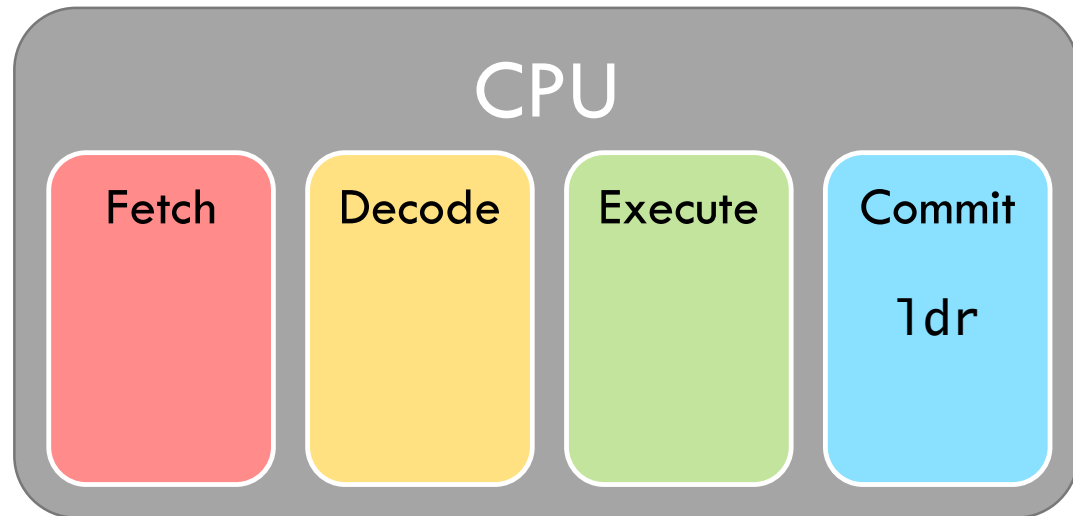
Evaluating polynomial on the simple CPU model

➔

```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3

ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3

...
```



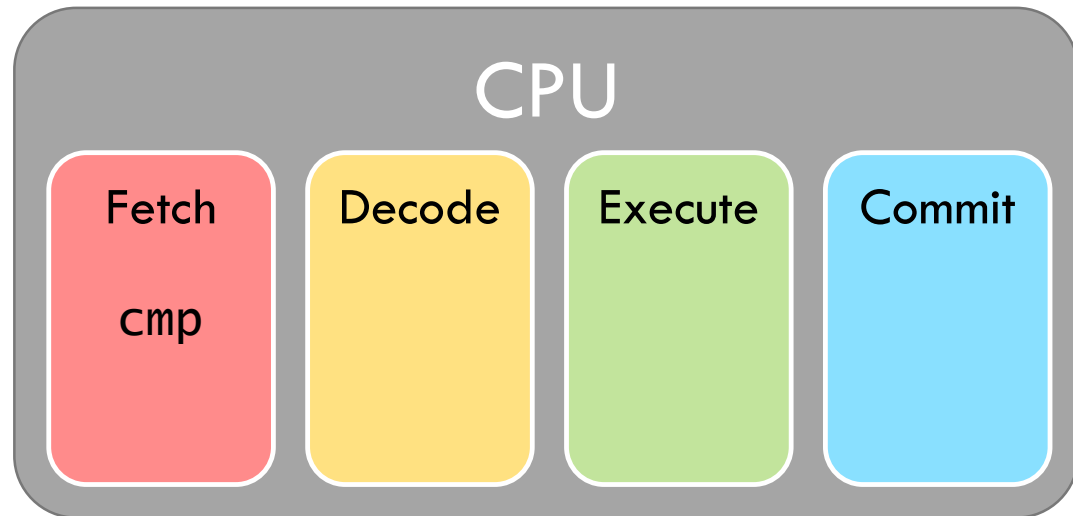
**4. Write values
into regs r5 and r3**

Evaluating polynomial on the simple CPU model

→ ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

...

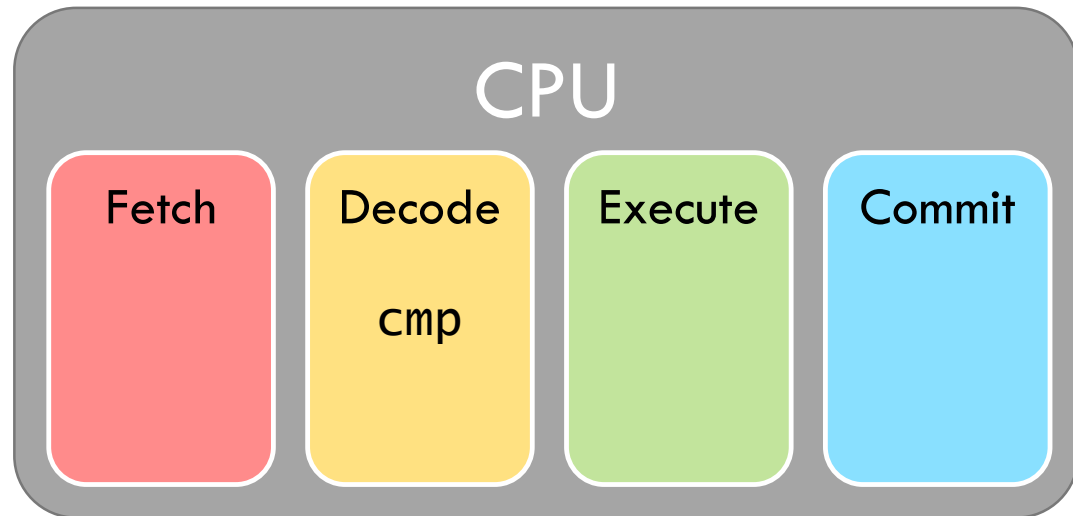


Evaluating polynomial on the simple CPU model

→ ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

...

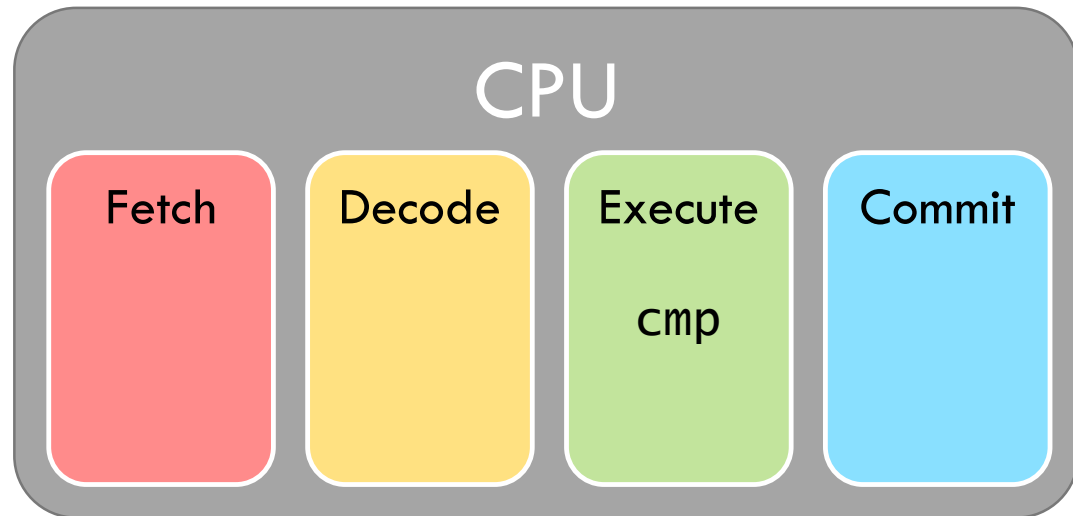


Evaluating polynomial on the simple CPU model

→ ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

...

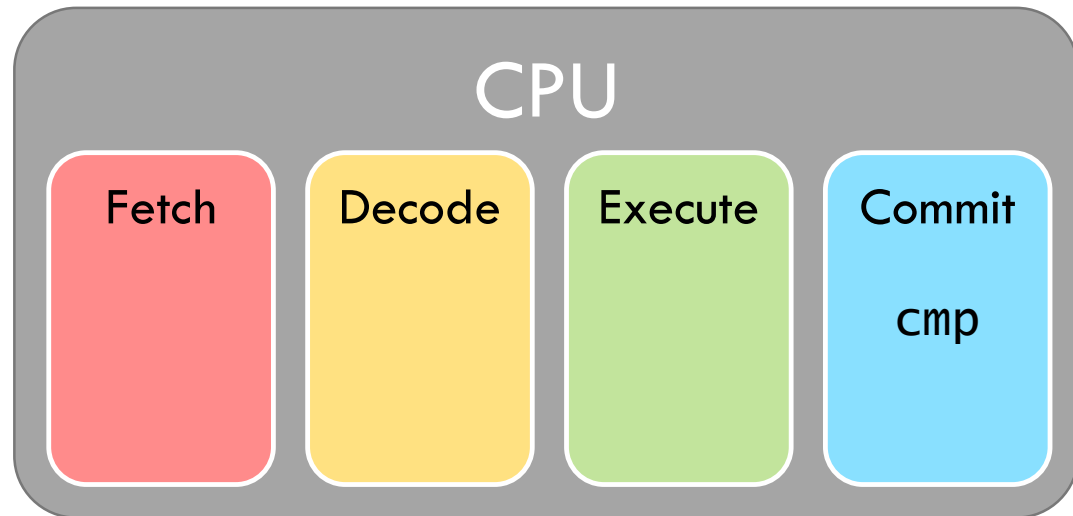


Evaluating polynomial on the simple CPU model

→ ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

...

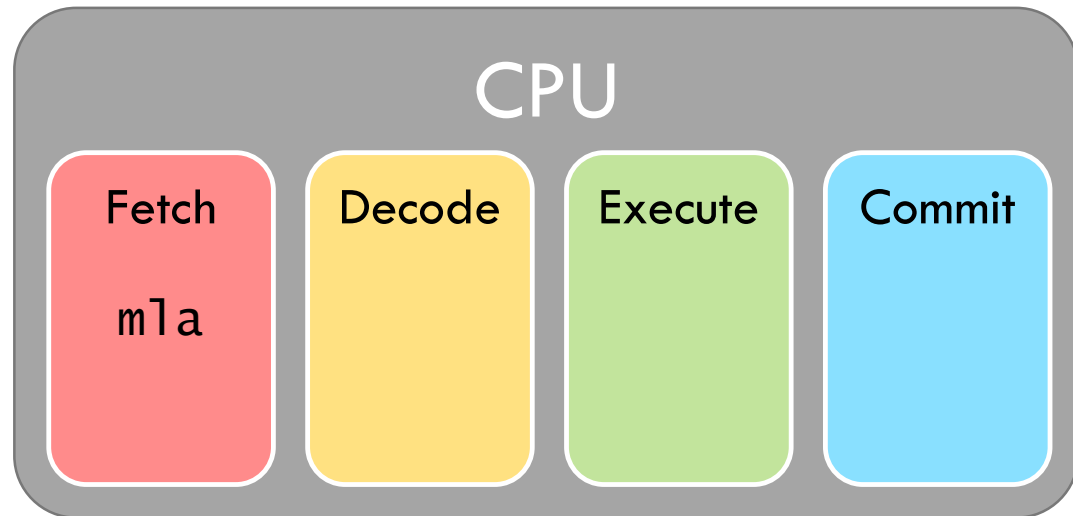


Evaluating polynomial on the simple CPU model

→
ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

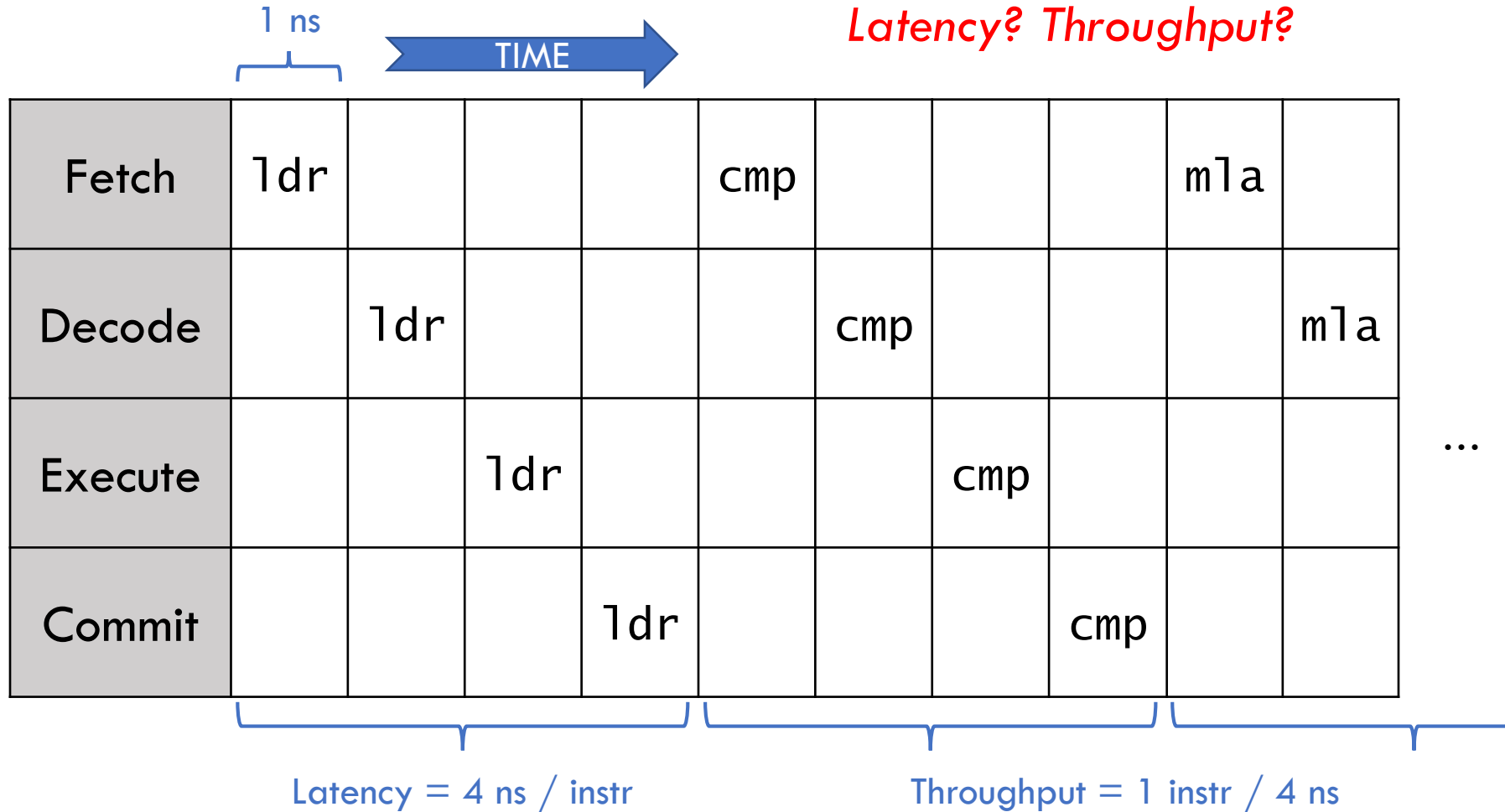
ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

...

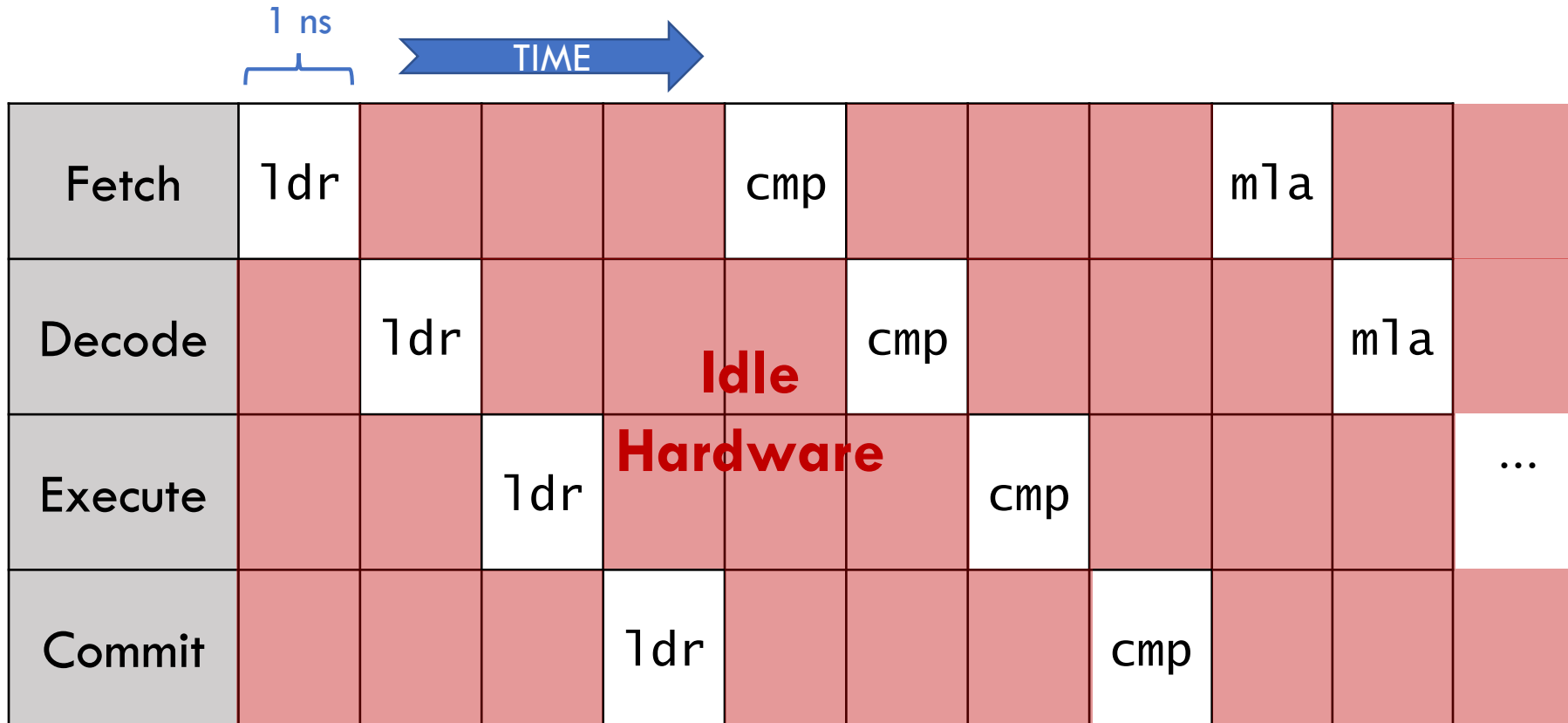


Evaluating polynomial on the simple CPU model

*How fast is this processor?
Latency? Throughput?*



Simple CPU is very wasteful



Pipelining

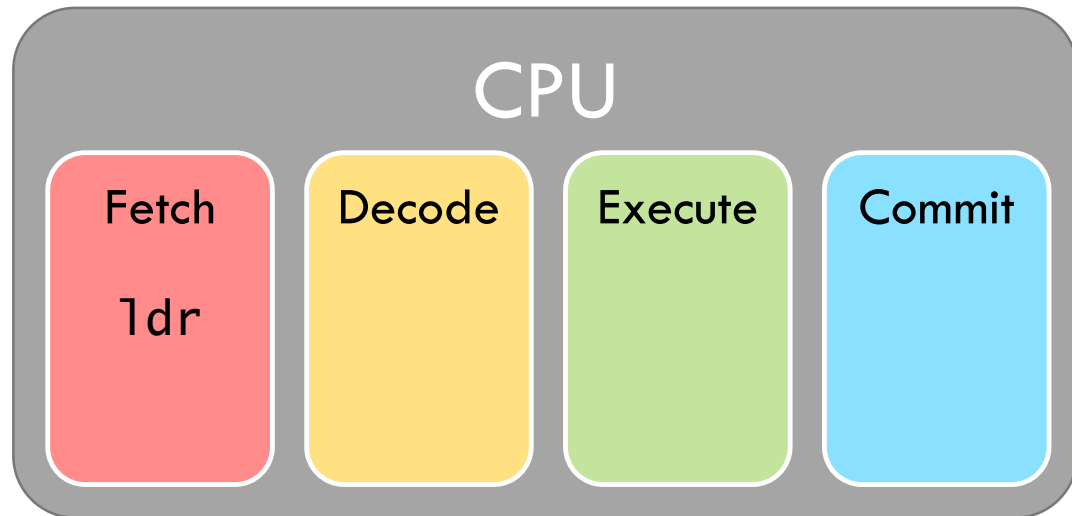
Pipelining keeps CPU busy through instruction-level parallelism

- Idea: Start on the next instr'n immediately

→
ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

...



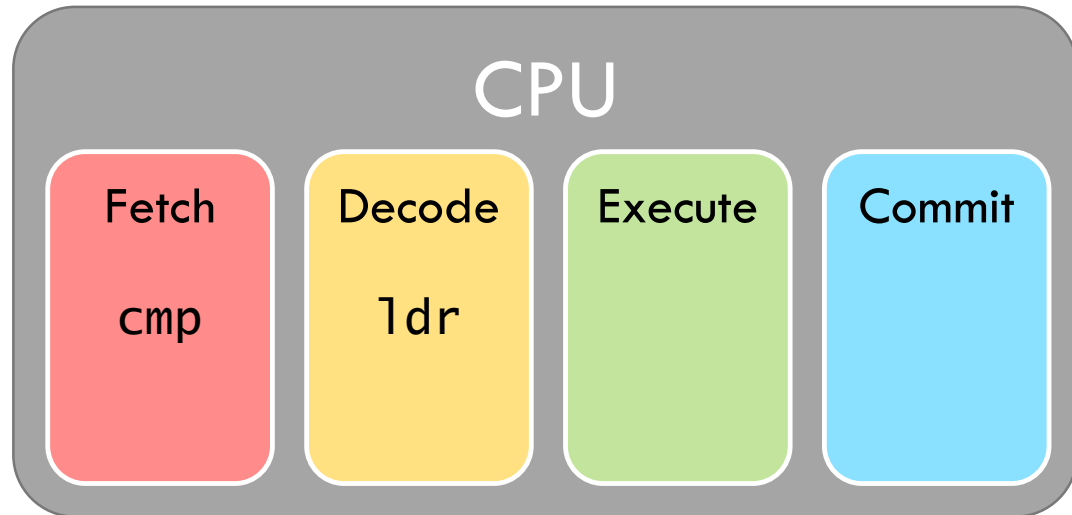
Pipelining keeps CPU busy through instruction-level parallelism

- Idea: Start on the next instr'n immediately

→
→
ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3

...



Pipelining keeps CPU busy through instruction-level parallelism

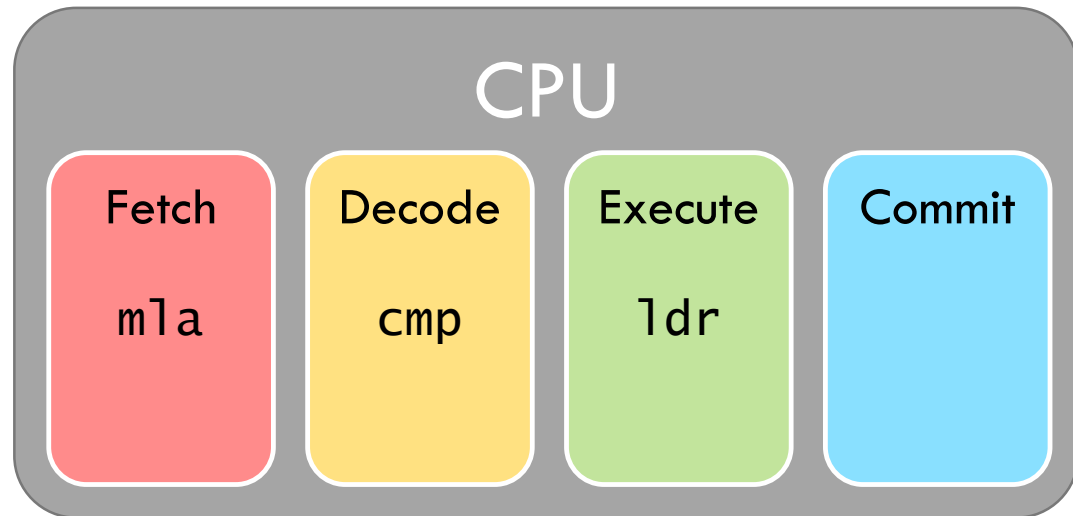
- Idea: Start on the next instr'n immediately



```
ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3

ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3

...
```



Pipelining keeps CPU busy through instruction-level parallelism

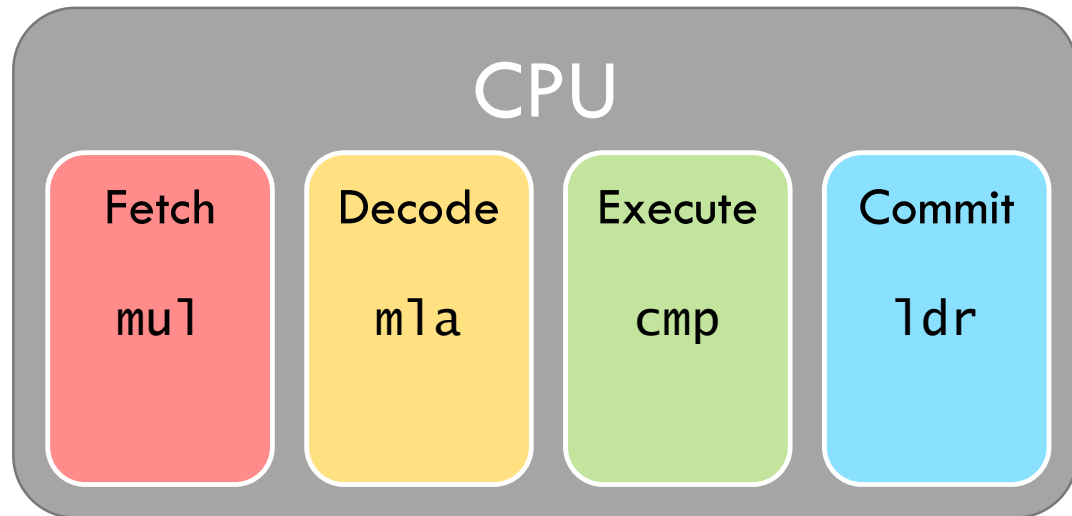
- Idea: Start on the next instr'n immediately



```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
```

```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
```

...



Pipelining keeps CPU busy through instruction-level parallelism

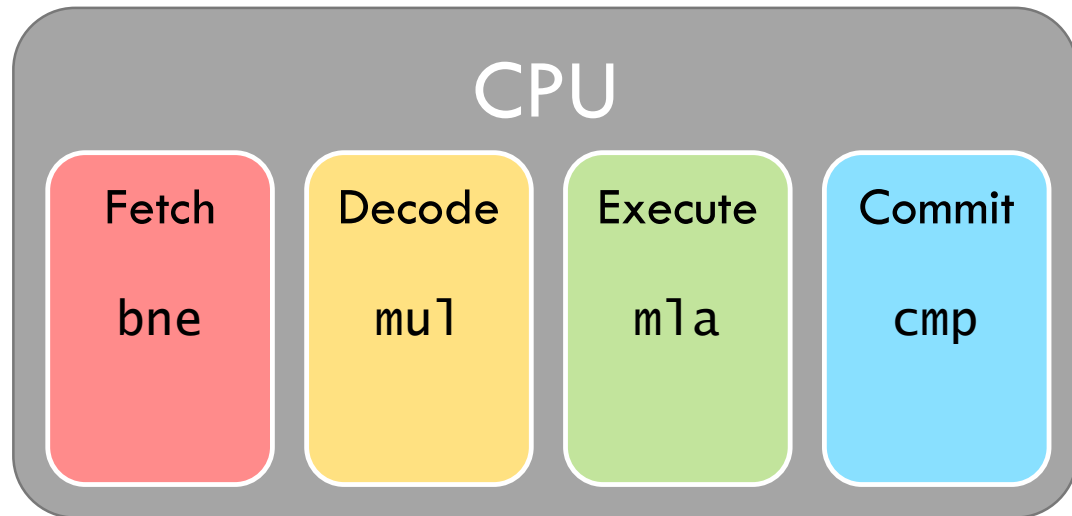
- Idea: Start on the next instr'n immediately



```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3

ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3

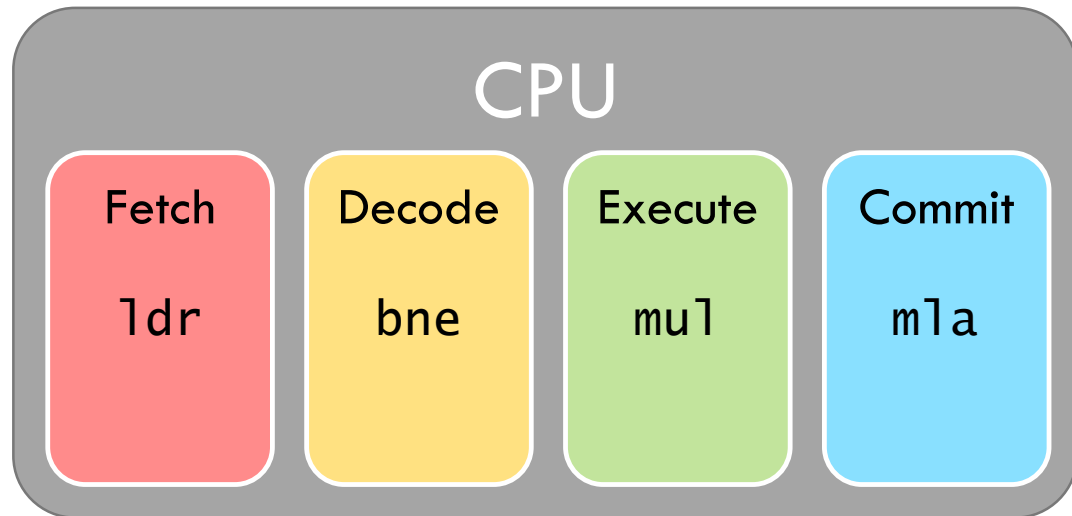
...
```



Pipelining keeps CPU busy through instruction-level parallelism

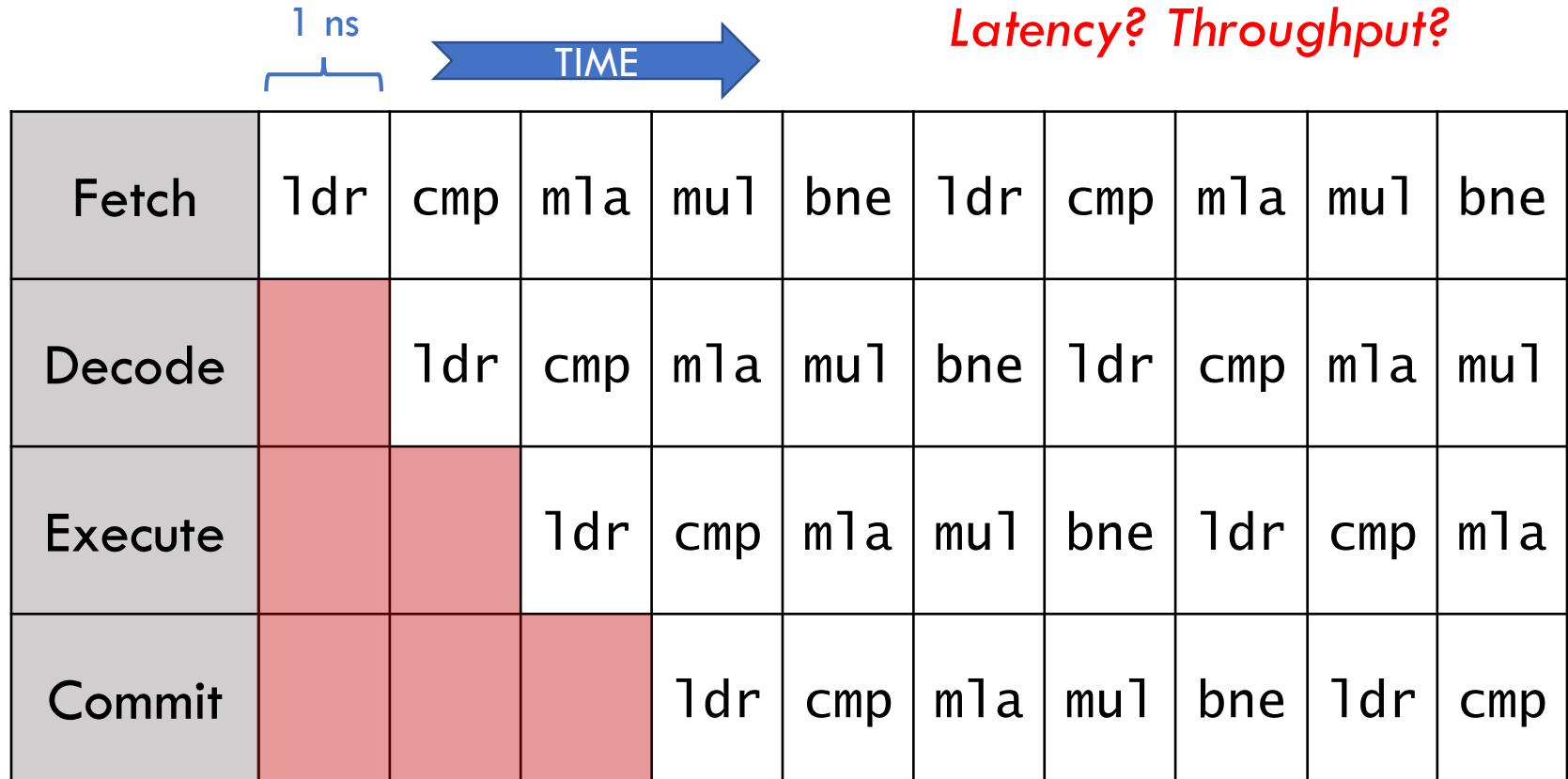
- Idea: Start on the next instr'n immediately

	ldr	r5, [r3], #4
	cmp	r1, r3
→	m1a	r0, r4, r5, r0
→	mul	r4, r2, r4
→	bne	.L3
→	ldr	r5, [r3], #4
	cmp	r1, r3
	m1a	r0, r4, r5, r0
	mul	r4, r2, r4
	bne	.L3
...		



Evaluating polynomial on the pipelined CPU

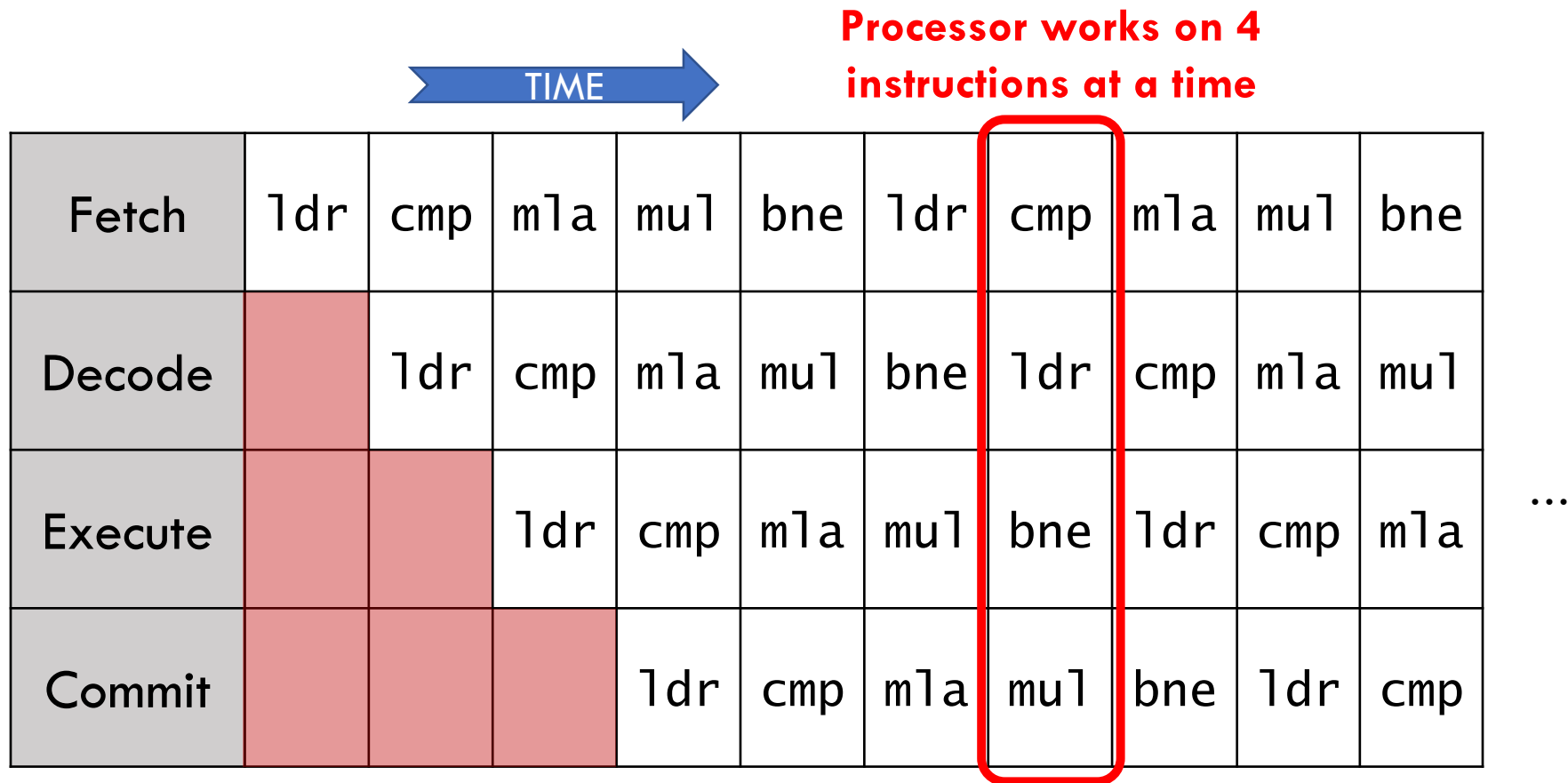
*How fast is this processor?
Latency? Throughput?*



Latency = 4 ns / instr

Throughput = 1 instr / ns
4X speedup!

Speedup achieved through pipeline parallelism



Limitations of pipelining

- Parallelism requires independent work
- Q: Are instructions independent?
- A: No! Many possible *hazards* limit parallelism...

Data hazards

```
ldr ra, [rb], #4 // ra ← Memory[rb]; rb ← rb + 4
cmp rc, rd        // rc ← rd + re
```

Q: When can the CPU pipeline the `cmp` behind `ldr`?

Fetch	ldr	cmp	...	...	...	...
Decode		ldr	cmp	...	...	...
Execute			ldr	cmp	...	...
Commit				ldr	cmp	...

- A: When they use **different registers**
 - Specifically, when `cmp` does not read any data written by `ldr`
 - E.g., `rb != rd`

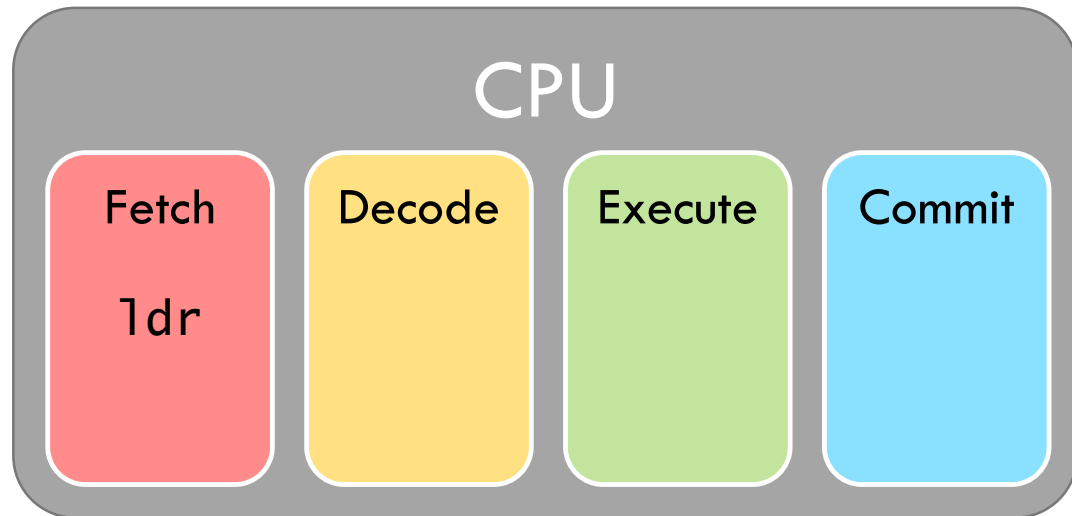
Dealing with data hazards: Stalling the pipeline

- Cannot pipeline `cmp` (`ldr` writes `r3`)

→ `ldr` `r5, [r3], #4`
`cmp` `r1, r3`
`mla` `r0, r4, r5, r0`
`mul` `r4, r2, r4`
`bne` `.L3`

`ldr` `r5, [r3], #4`
`cmp` `r1, r3`
`mla` `r0, r4, r5, r0`
`mul` `r4, r2, r4`
`bne` `.L3`

...



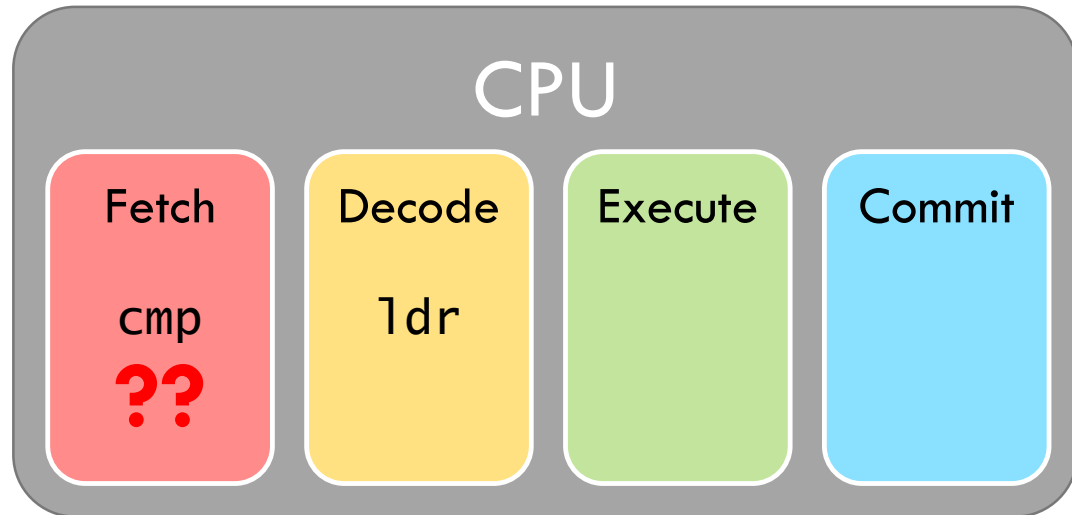
Dealing with data hazards: Stalling the pipeline

- Cannot pipeline `cmp` (`ldr` writes `r3`)

→ `ldr` `r5, [r3], #4`
→ `cmp` `r1, r3`
`mla` `r0, r4, r5, r0`
`mul` `r4, r2, r4`
`bne` `.L3`

`ldr` `r5, [r3], #4`
`cmp` `r1, r3`
`mla` `r0, r4, r5, r0`
`mul` `r4, r2, r4`
`bne` `.L3`

...



Dealing with data hazards: Stalling the pipeline

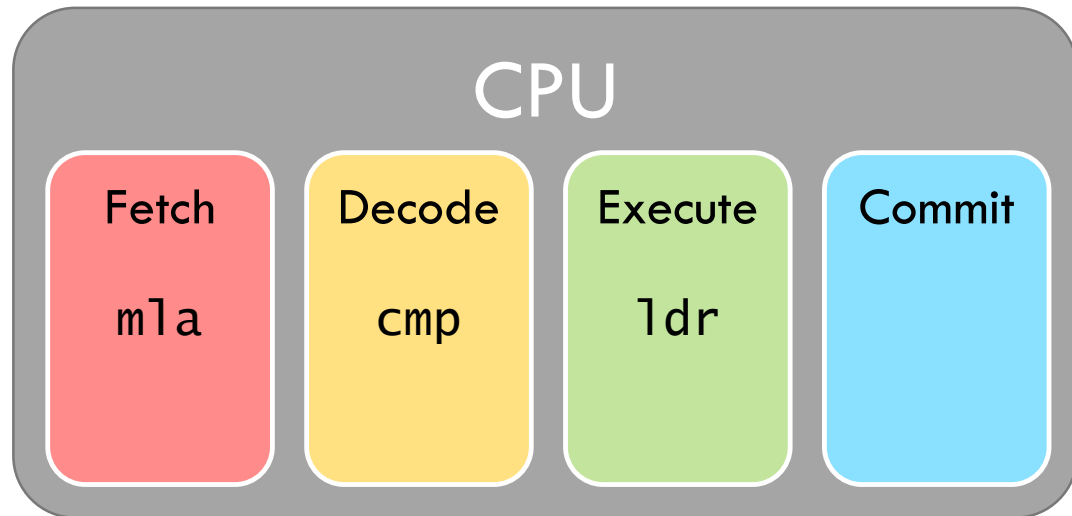
- Cannot pipeline `cmp` (`ldr` writes `r3`)



```
ldr    r5, [r3], #4  
cmp    r1, r3  
mla    r0, r4, r5, r0  
mul    r4, r2, r4  
bne    .L3
```

```
ldr    r5, [r3], #4  
cmp    r1, r3  
mla    r0, r4, r5, r0  
mul    r4, r2, r4  
bne    .L3
```

...



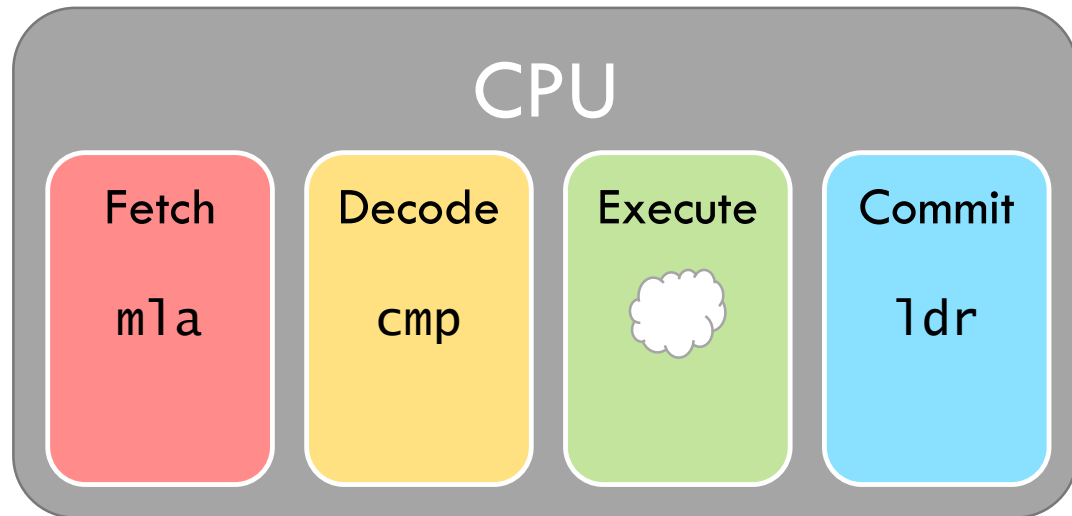
Dealing with data hazards: Stalling the pipeline

- Cannot pipeline `cmp` (`ldr` writes `r3`)

→ `ldr` `r5, [r3], #4`
→ `cmp` `r1, r3`
→ `m1a` `r0, r4, r5, r0`
`mul` `r4, r2, r4`
`bne` `.L3`

`ldr` `r5, [r3], #4`
`cmp` `r1, r3`
`m1a` `r0, r4, r5, r0`
`mul` `r4, r2, r4`
`bne` `.L3`

...



**Inject a “bubble” (NOP)
into the pipeline**

Dealing with data hazards: Stalling the pipeline

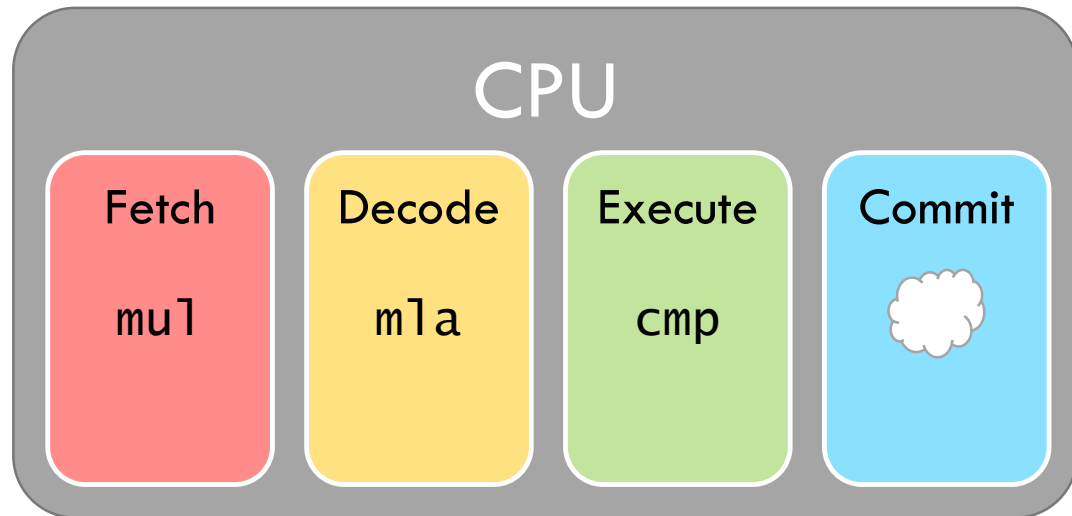
- Cannot pipeline `cmp` (`ldr` writes `r3`)



```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
```

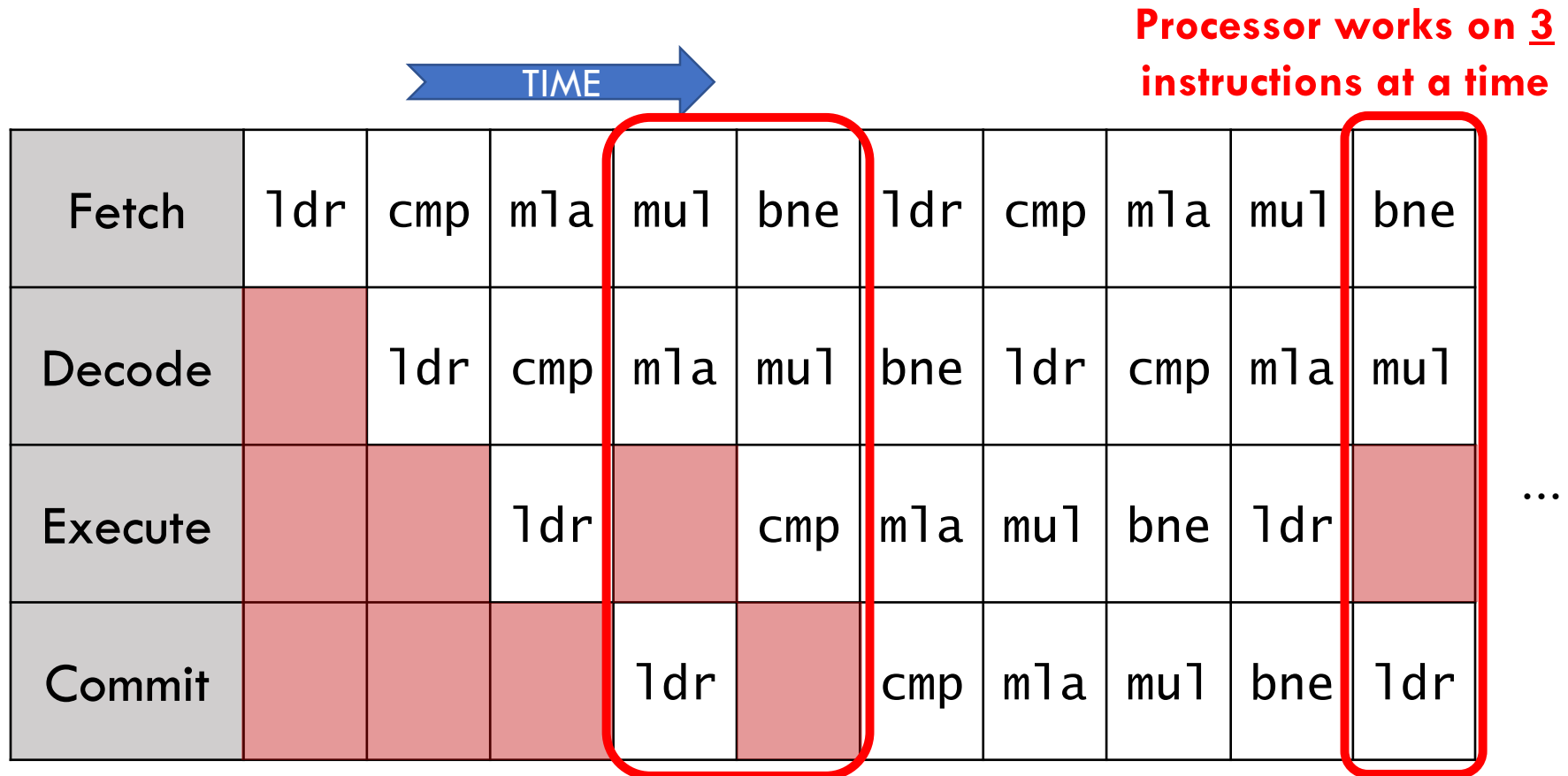
```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
```

...



**cmp proceeds once `ldr`
has committed**

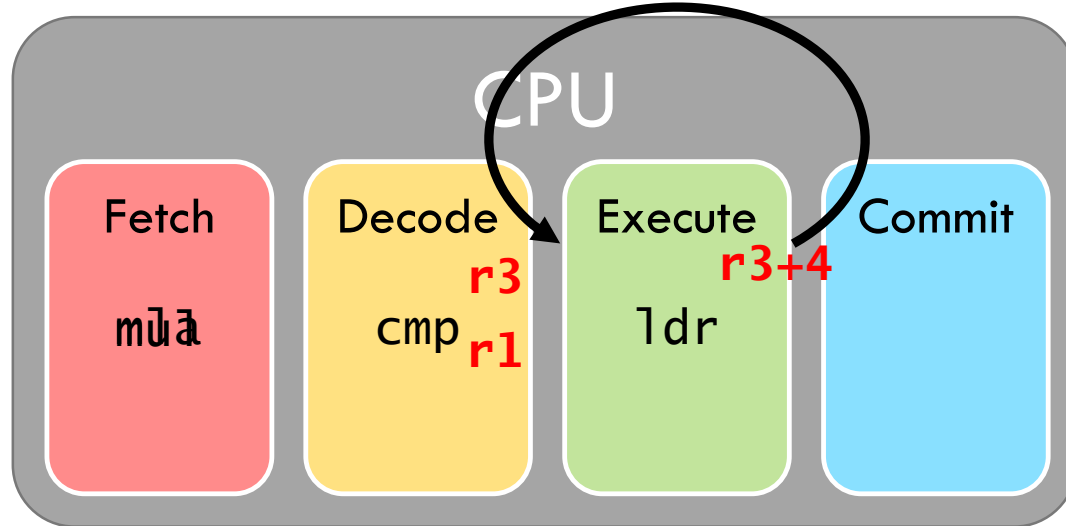
Stalling degrades performance



- But stalling is sometimes unavoidable
 - E.g., long-latency instructions (divide, cache miss)

Dealing with data hazards: Forwarding data

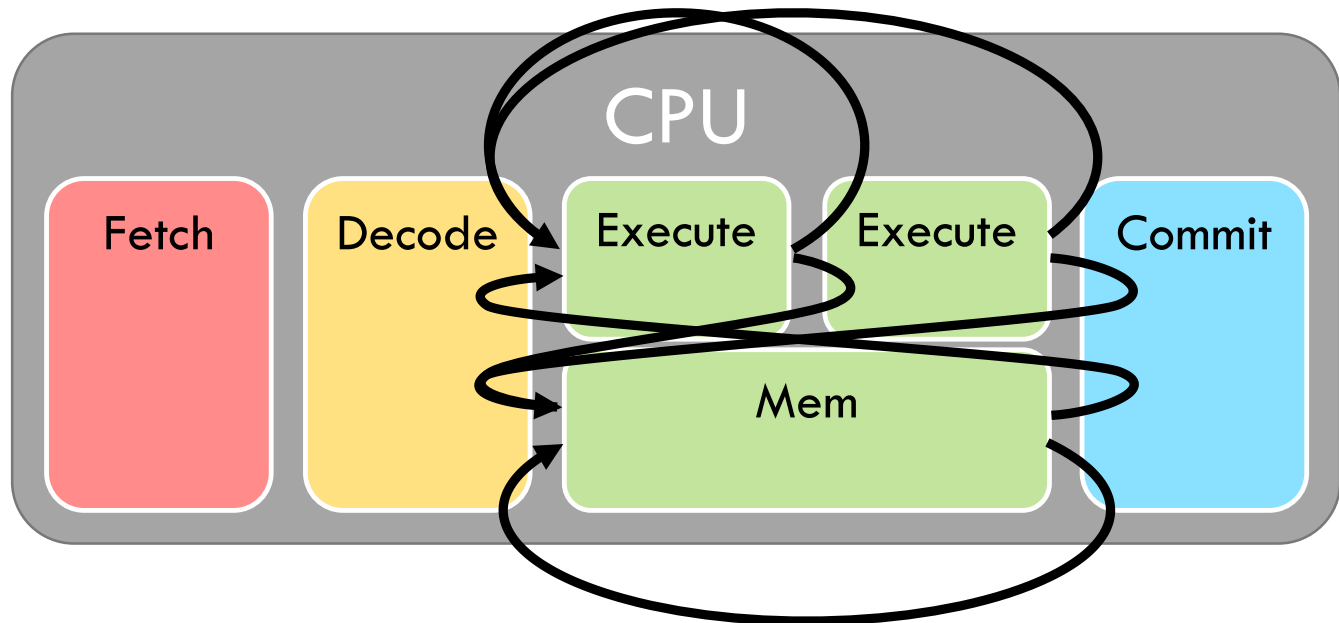
- Wait a second... data is available after Execute!



- Forwarding eliminates many (not all) pipeline stalls

Pipelining is not free!

- Q: How well does forwarding scale?
- A: Not well... many forwarding paths in deep & complex pipelines

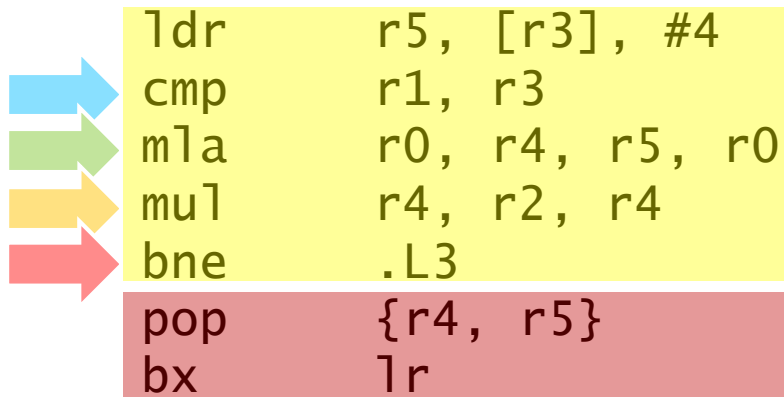


Control hazards + Speculation

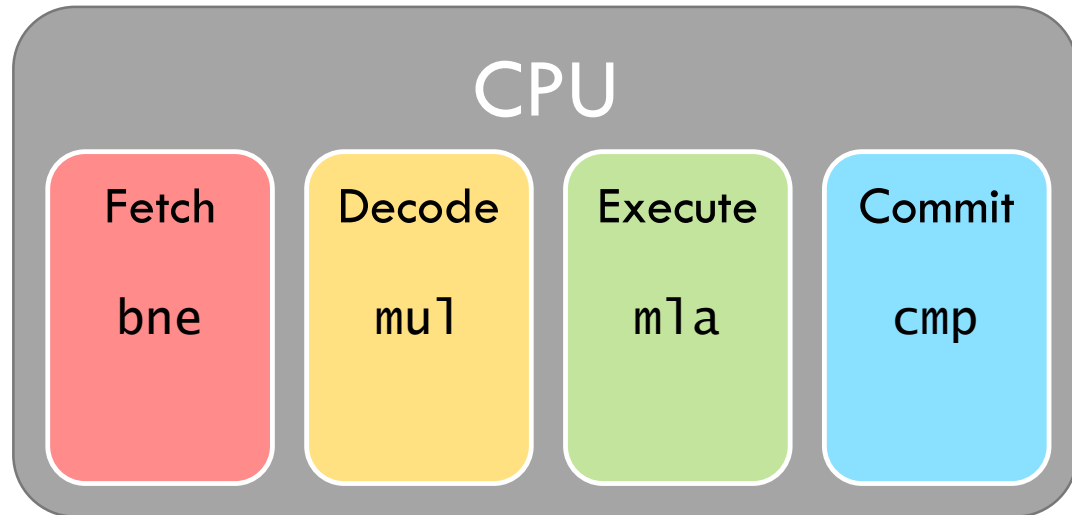
- Programs must appear to execute *in program order*
 - ➔ All instructions depend on earlier ones
- Most instructions implicitly continue at the next...
- But **branches** redirect execution to new location

Dealing with control hazards: Flushing the pipeline

- What if we always fetch the next instruction?

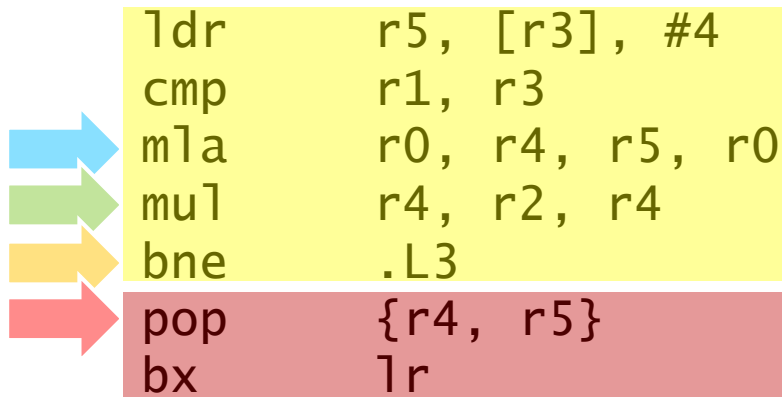


Static instruction sequence
(i.e., program layout in memory)

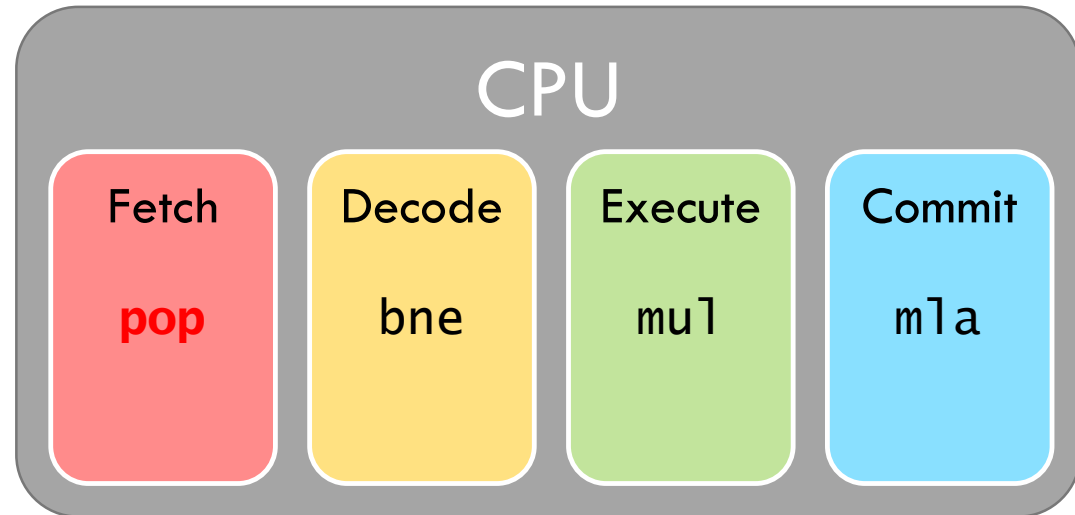


Dealing with control hazards: Flushing the pipeline

- What if we always fetch the next instruction?

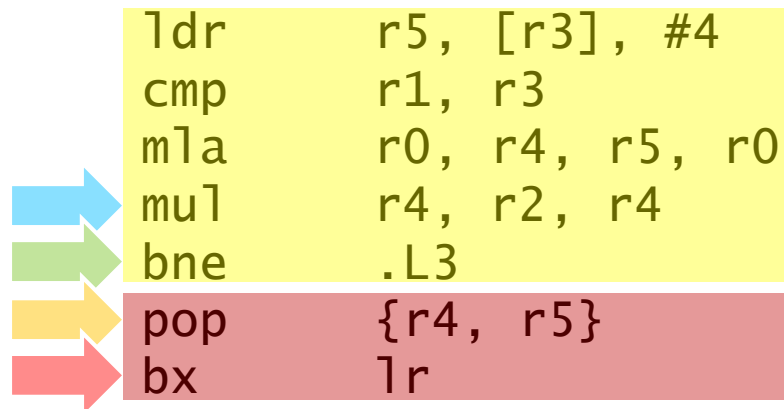


Static instruction sequence
(i.e., program layout in memory)

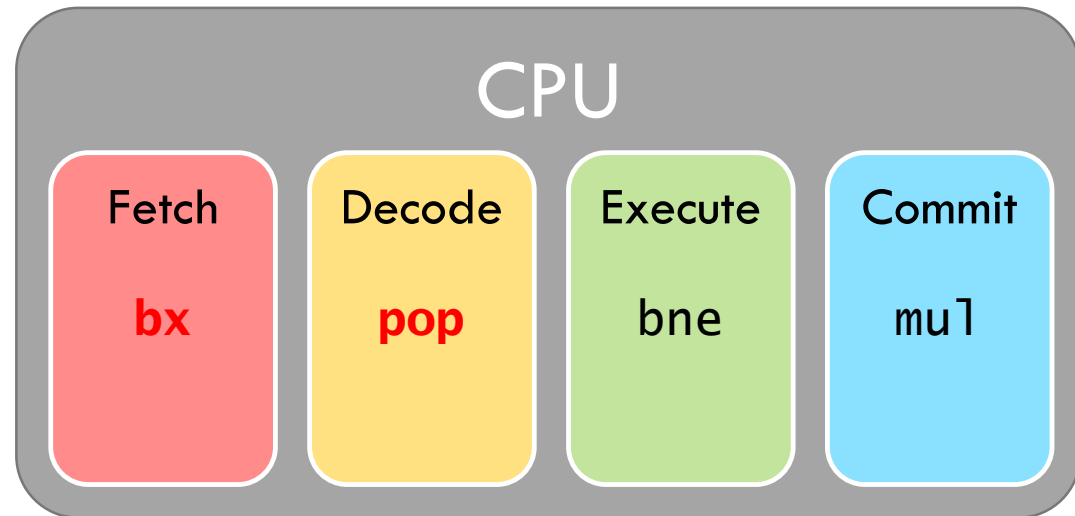


Dealing with control hazards: Flushing the pipeline

- What if we always fetch the next instruction?



Static instruction sequence
(i.e., program layout in memory)

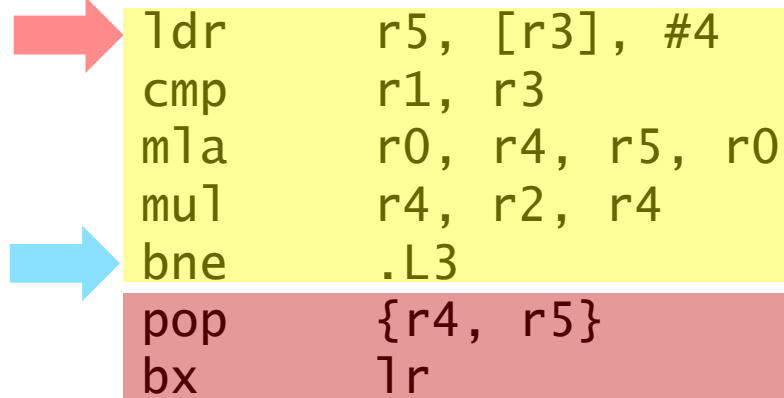


**Whoops! We fetched the
wrong instructions!
(Loop not finished)**

Dealing with control hazards: Flushing the pipeline

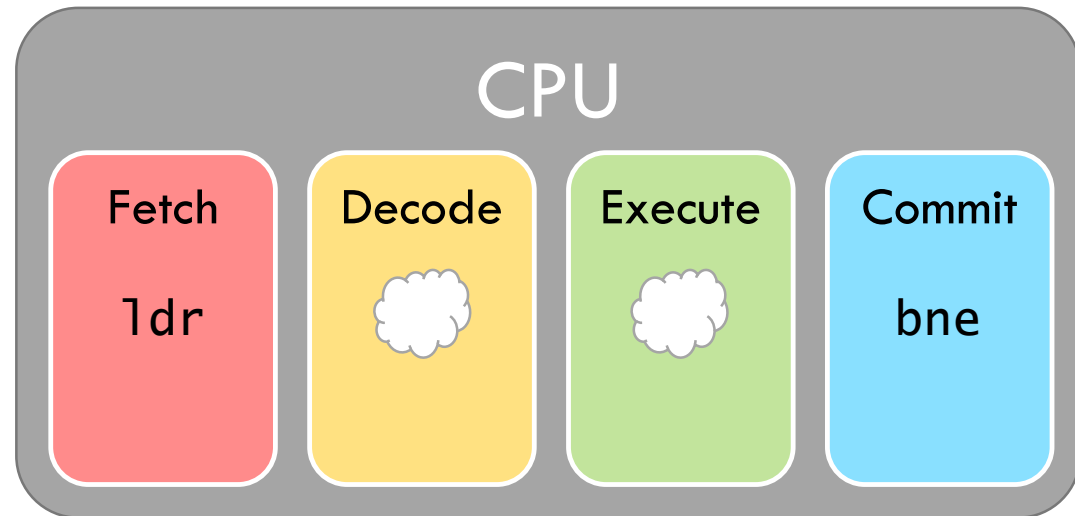
(Next loop
iteration)

- What if we always fetch the next instruction?



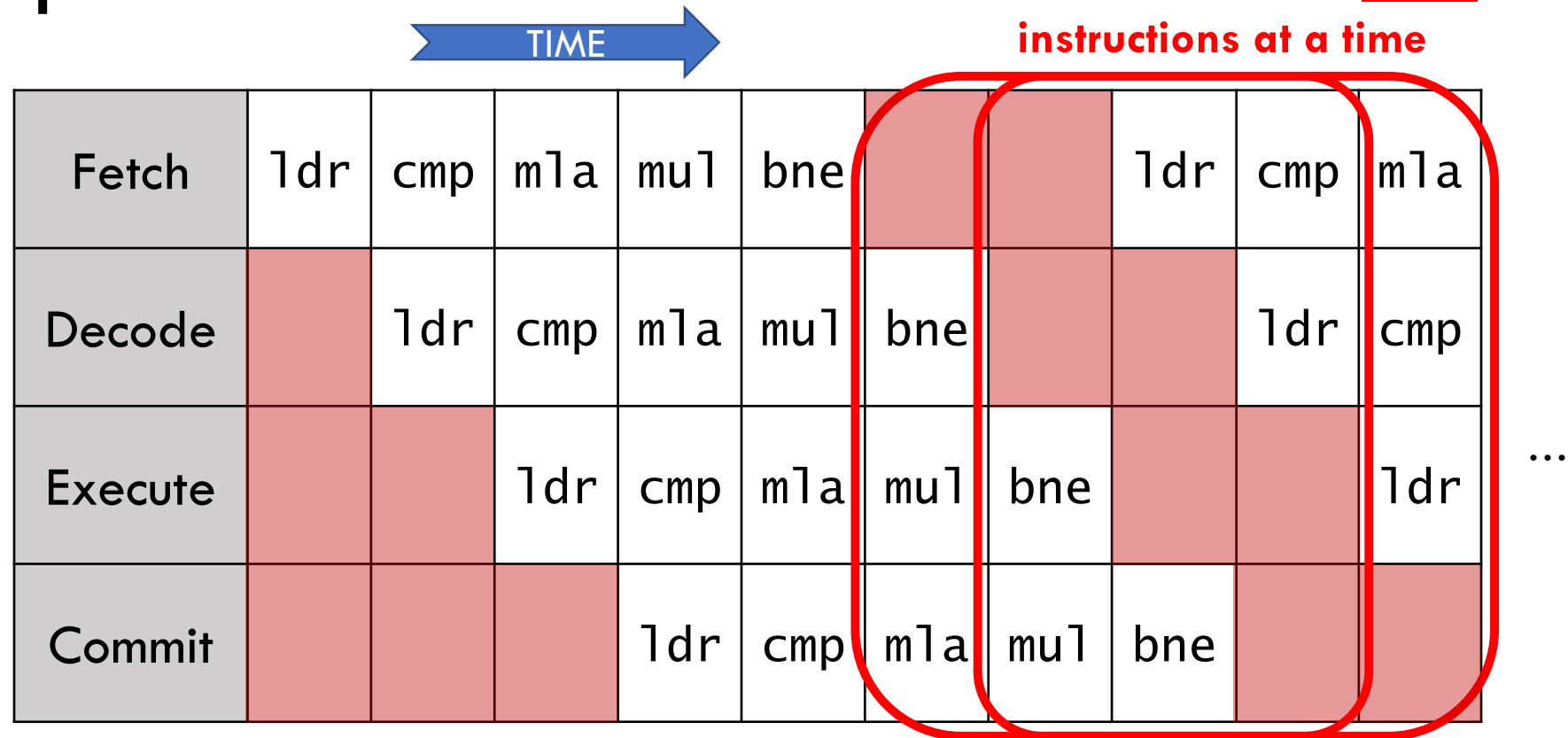
```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
pop    {r4, r5}
bx     lr
```

Static instruction sequence
(i.e., program layout in memory)



**Whoops! We fetched the
wrong instructions!
(Loop not finished)**

Pipeline flushes destroy performance



- Penalty increases with deeper pipelines

Dealing with control hazards: *Speculation!*

- Processors do not wait for branches to execute
- Instead, they speculate (i.e., guess) where to go next + start fetching
- Modern processors use very sophisticated mechanisms
 - E.g., speculate in Fetch stage—before processor even knows instrn is a branch!
 - >95% prediction accuracy
 - Still, branch mis-speculation is major problem

Pipelining Summary

- Pipelining is a simple, effective way to improve throughput
 - N -stage pipeline gives up to $N \times$ speedup
 - Pipelining has limits
 - Hard to keep pipeline busy because of hazards
 - Forwarding is expensive in deep pipelines
 - Pipeline flushes are expensive in deep pipelines
- ➔ Pipelining is ubiquitous, but tops out at $N \approx 15$

Out-of-Order Execution

Increasing parallelism via dataflow

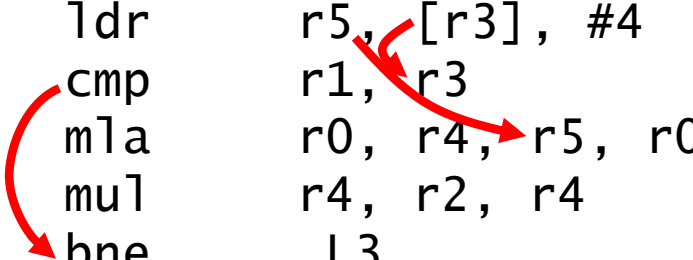
- Parallelism limited by many *false dependencies*, particularly *sequential program order*
- **Dataflow** tracks how instructions actually depend on each other
 - *True dependence*: read-after-write

Dataflow increases parallelism by eliminating unnecessary dependences

Example: Dataflow in polynomial evaluation

1dr
cmp
mla
mul
bne

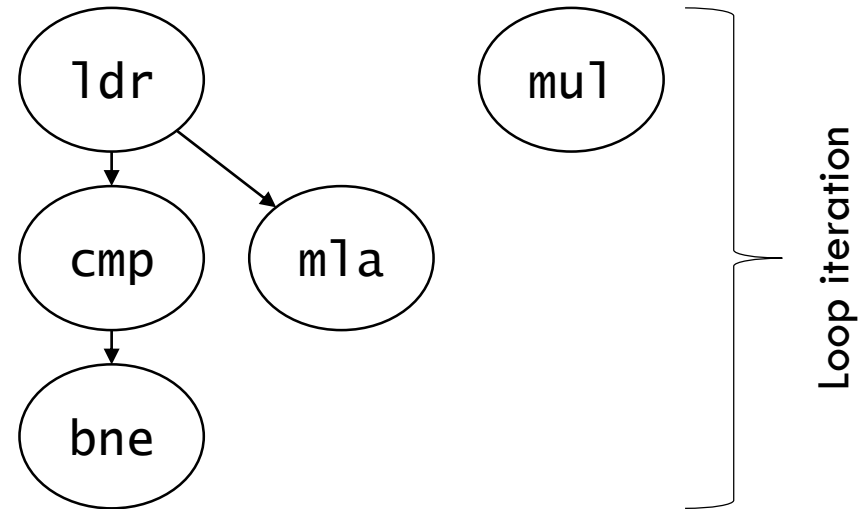
r5, [r3], #4
r1, r3
r0, r4, r5, r0
r4, r2, r4
.L3



1dr
cmp
mla
mul
bne

r5, [r3], #4
r1, r3
r0, r4, r5, r0
r4, r2, r4
.L3

...

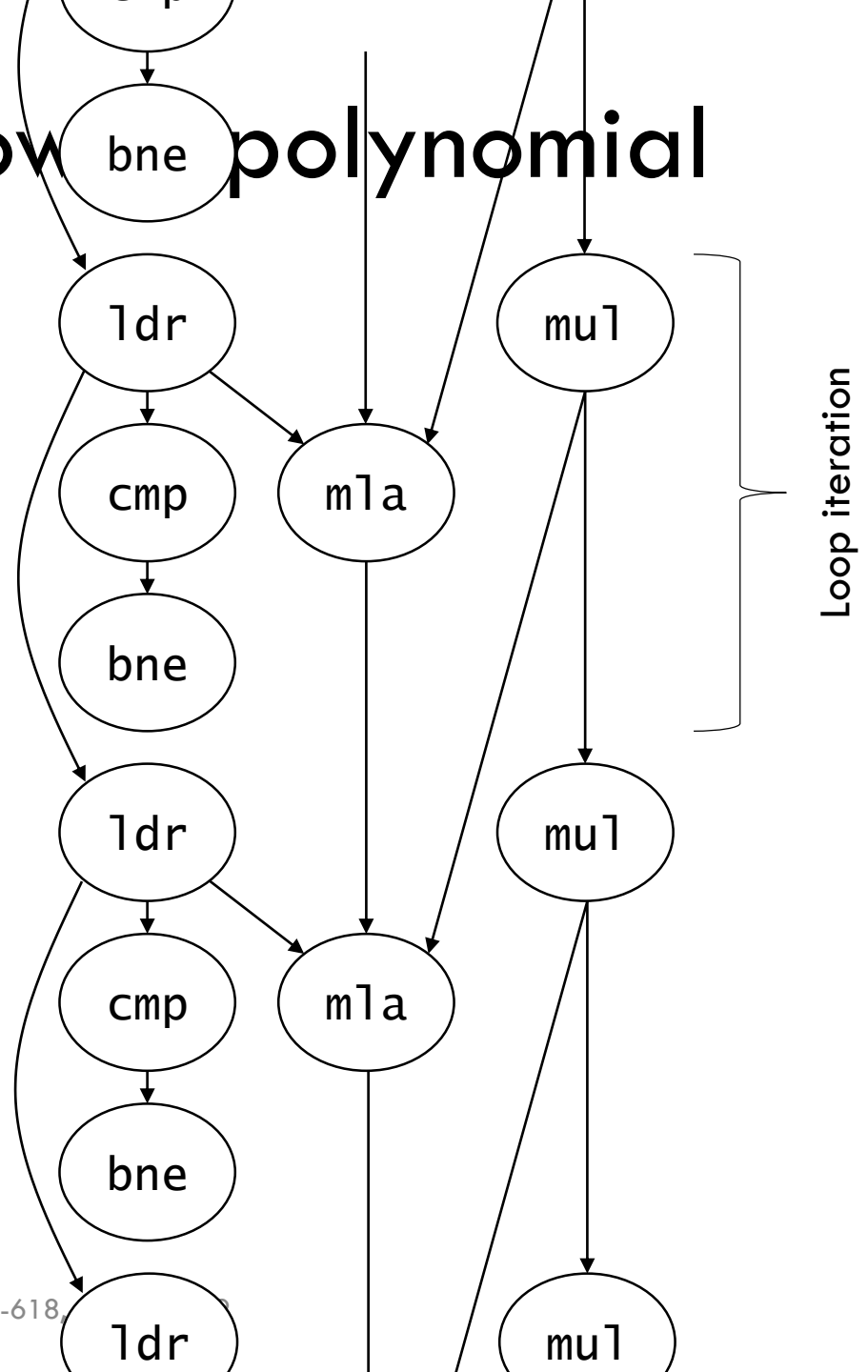


Example: Dataflow polynomial evaluation

```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
```

```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
```

...



Example: Dataflow polynomial execution

- Execution only, with perfect scheduling & unlimited execution units
 - ldr, mul execute in 2 cycles
 - cmp, bne execute in 1 cycle
 - mla executes in 3 cycles
- Q: Does dataflow speedup execution? By how much?
- Q: What is the performance bottleneck?



1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

ldr

```
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
```



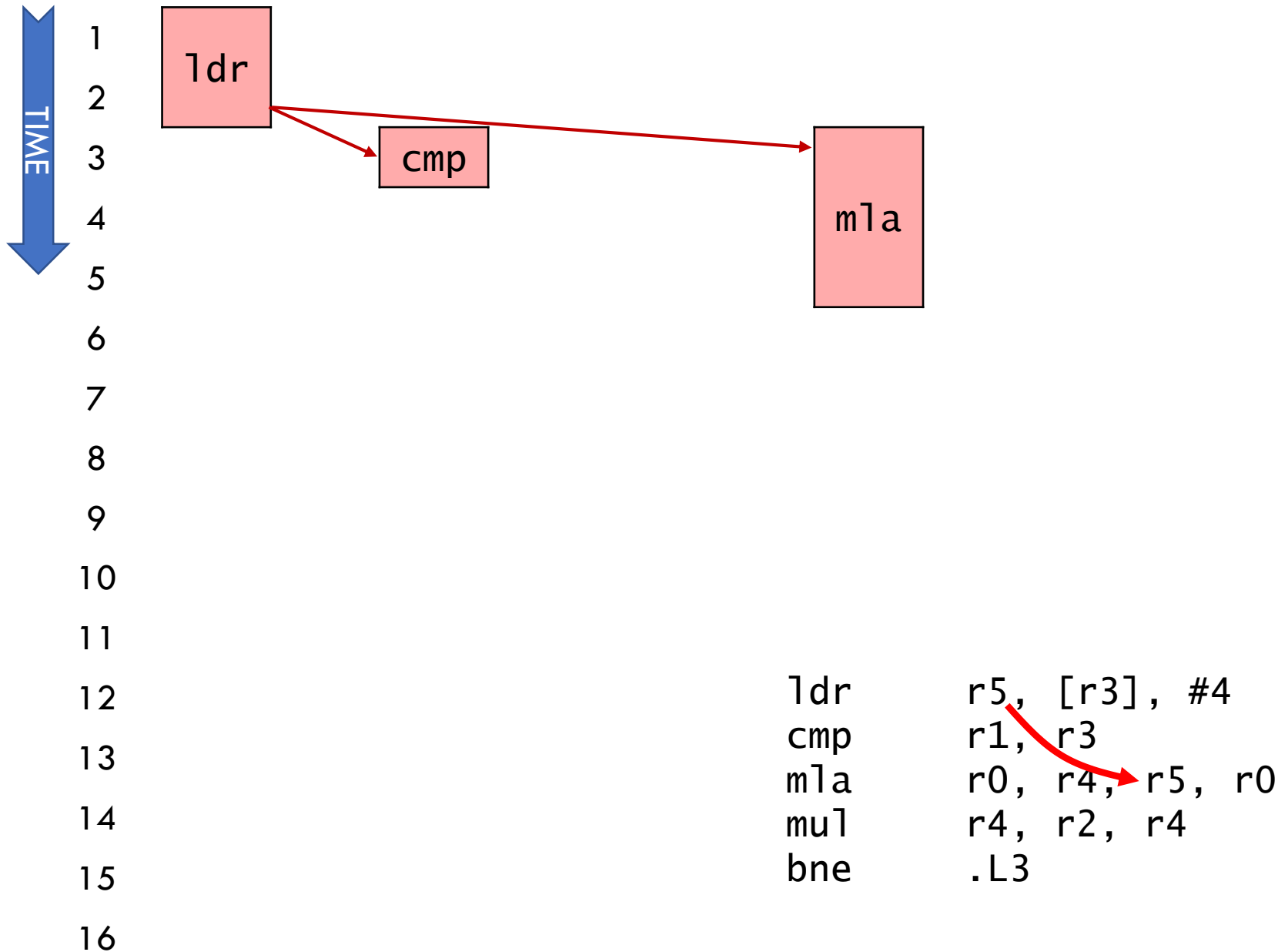
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

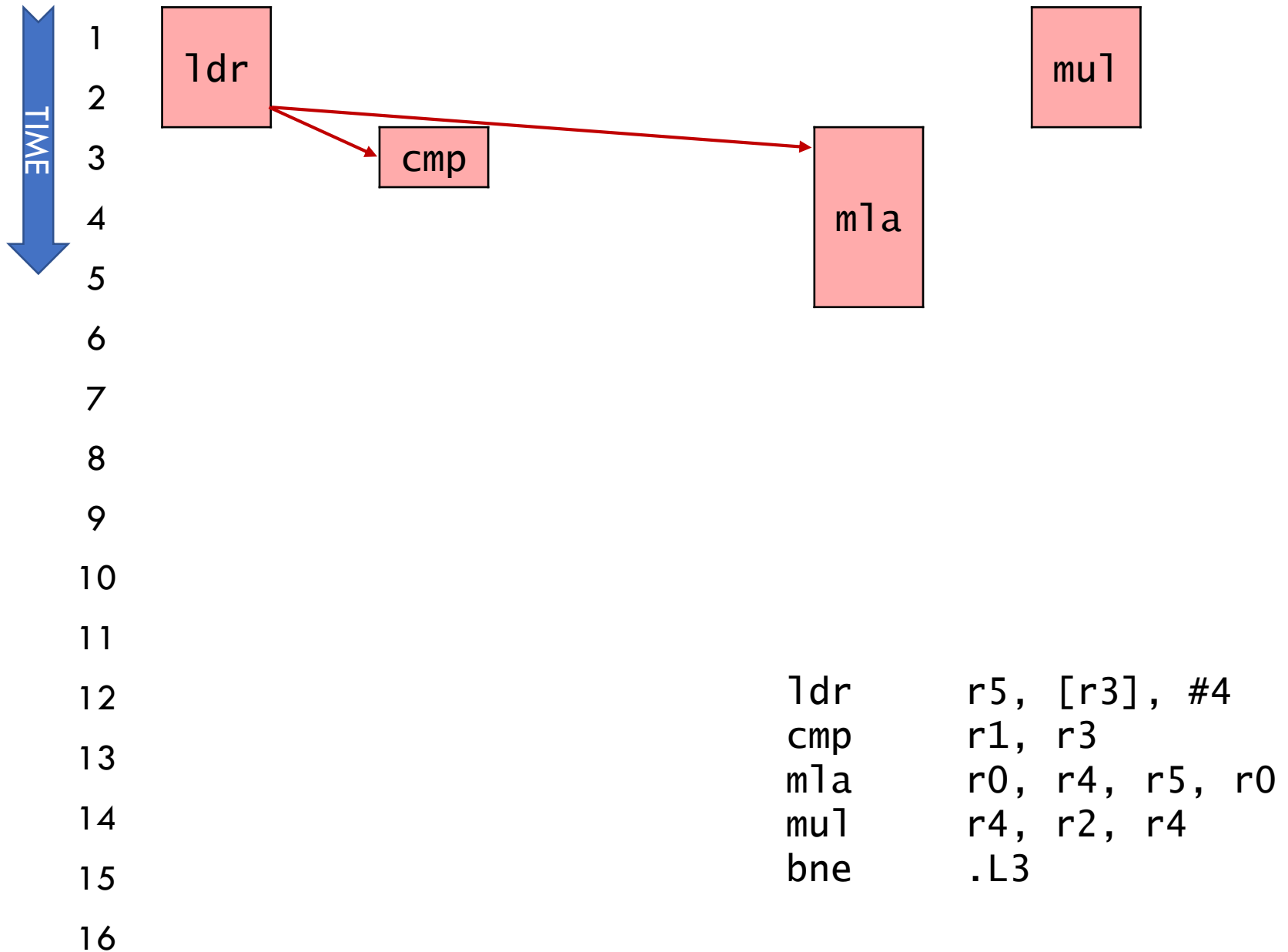
ldr

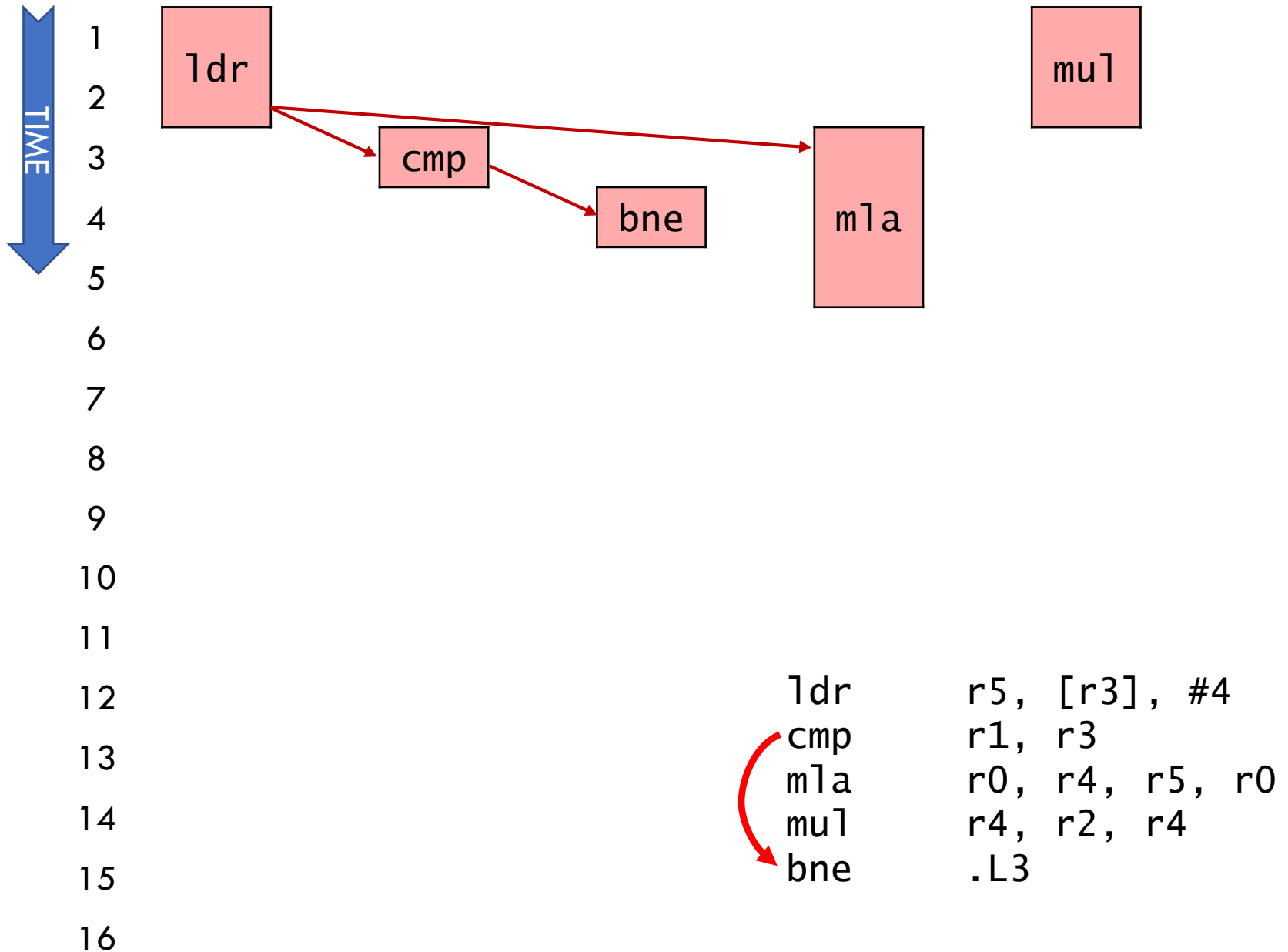


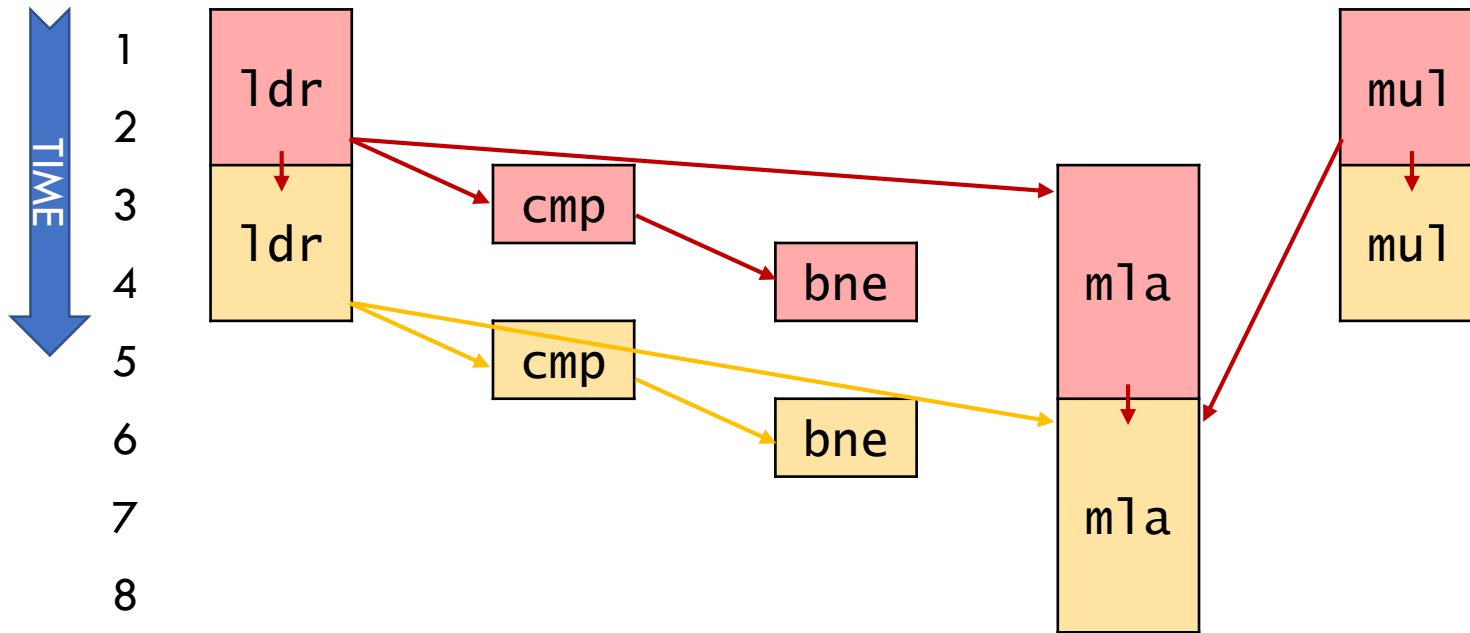
cmp

```
ldr    r5, [r3], #4  
cmp    r1, r3  
mla    r0, r4, r5, r0  
mul    r4, r2, r4  
bne    .L3
```





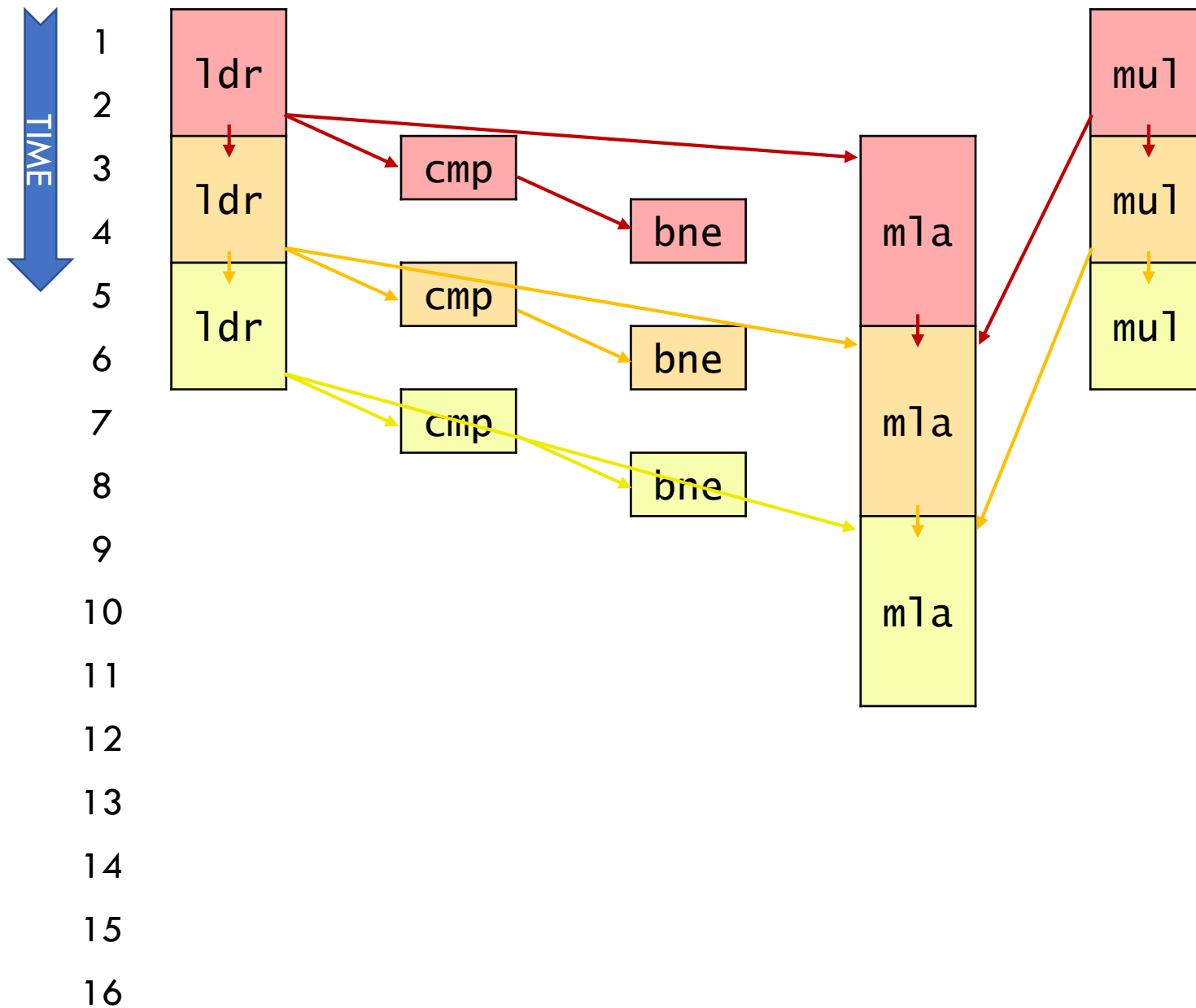


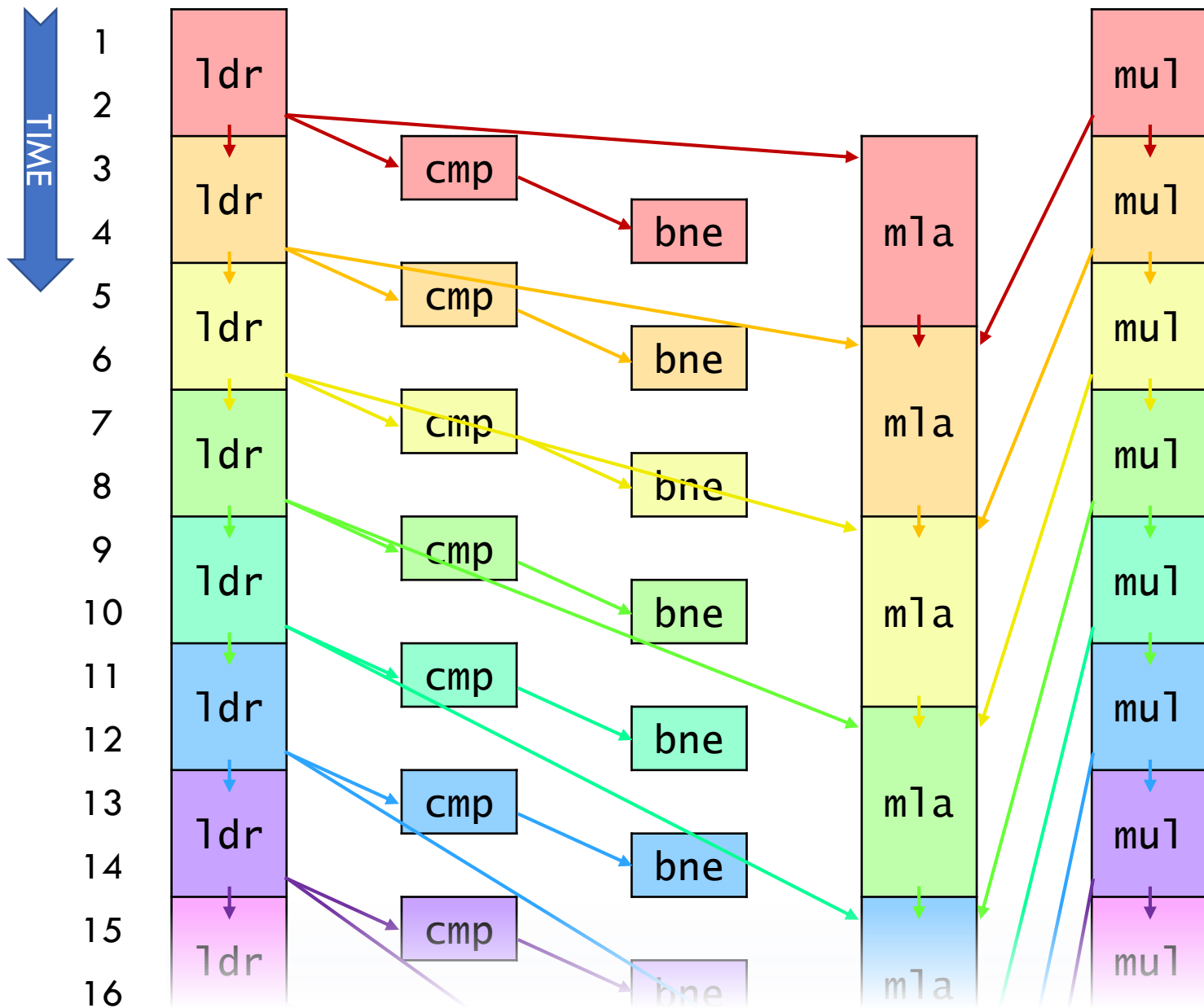


```

10  ldr    r5, [r3], #4
11  cmp    r1, r3
12  mla    r0, r4, r5, r0
13  mul    r4, r2, r4
14  bne    .L3
15  ldr    r5, [r3], #4
16  cmp    r1, r3
17  mla    r0, r4, r5, r0
18  mul    r4, r2, r4
19  bne    .L3

```





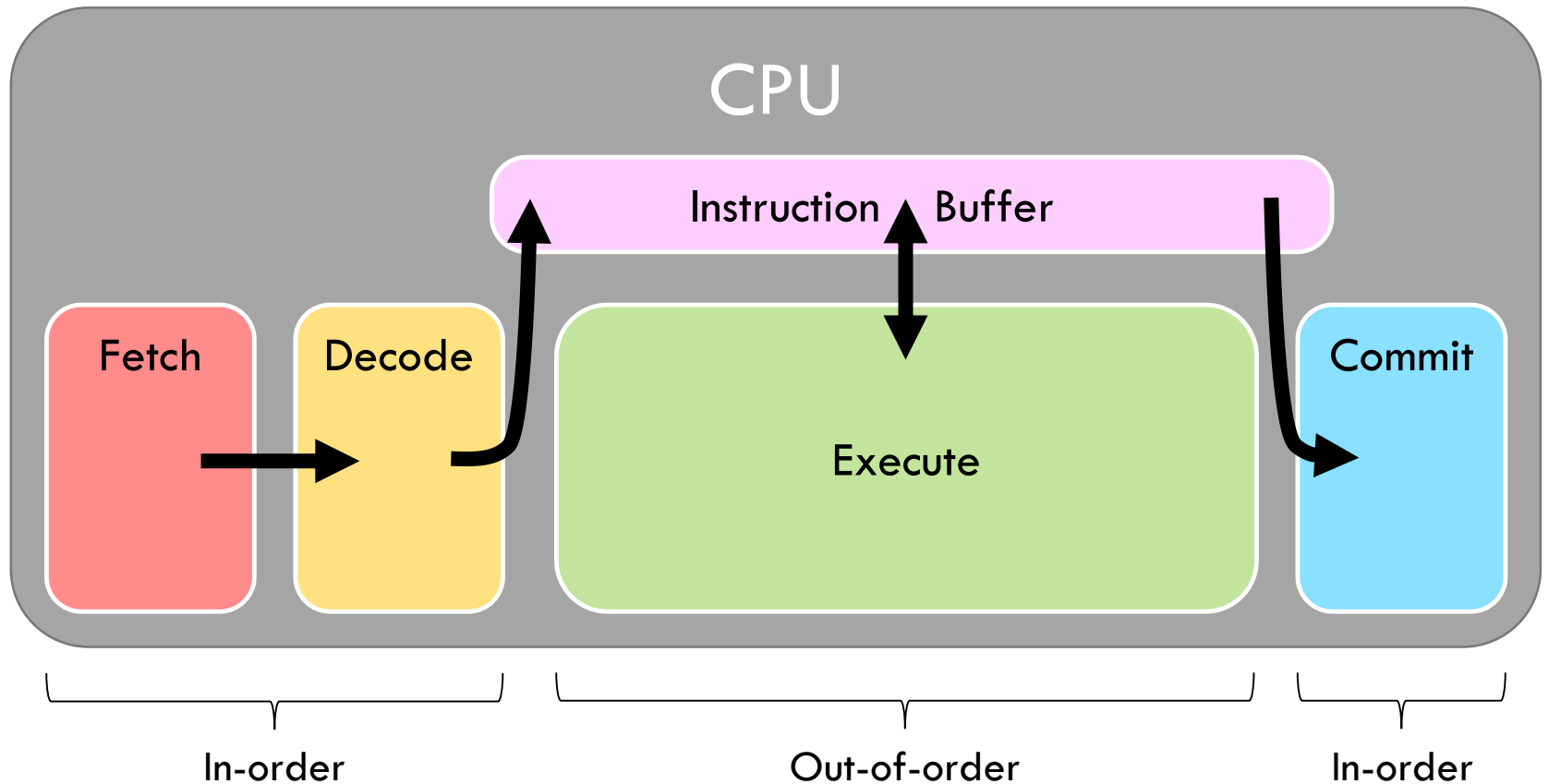
Example: Dataflow polynomial execution

- Q: Does dataflow speedup execution? By how much?
 - Yes! 3 cycles / loop iteration
 - Instructions per cycle (IPC) = $5/3 \approx 1.67$
(vs. 1 for perfect pipelining)
- Q: What is the performance bottleneck?
 - m1a: Each m1a depends on previous m1a & takes 3 cycles
 - → This program is **latency-bound**

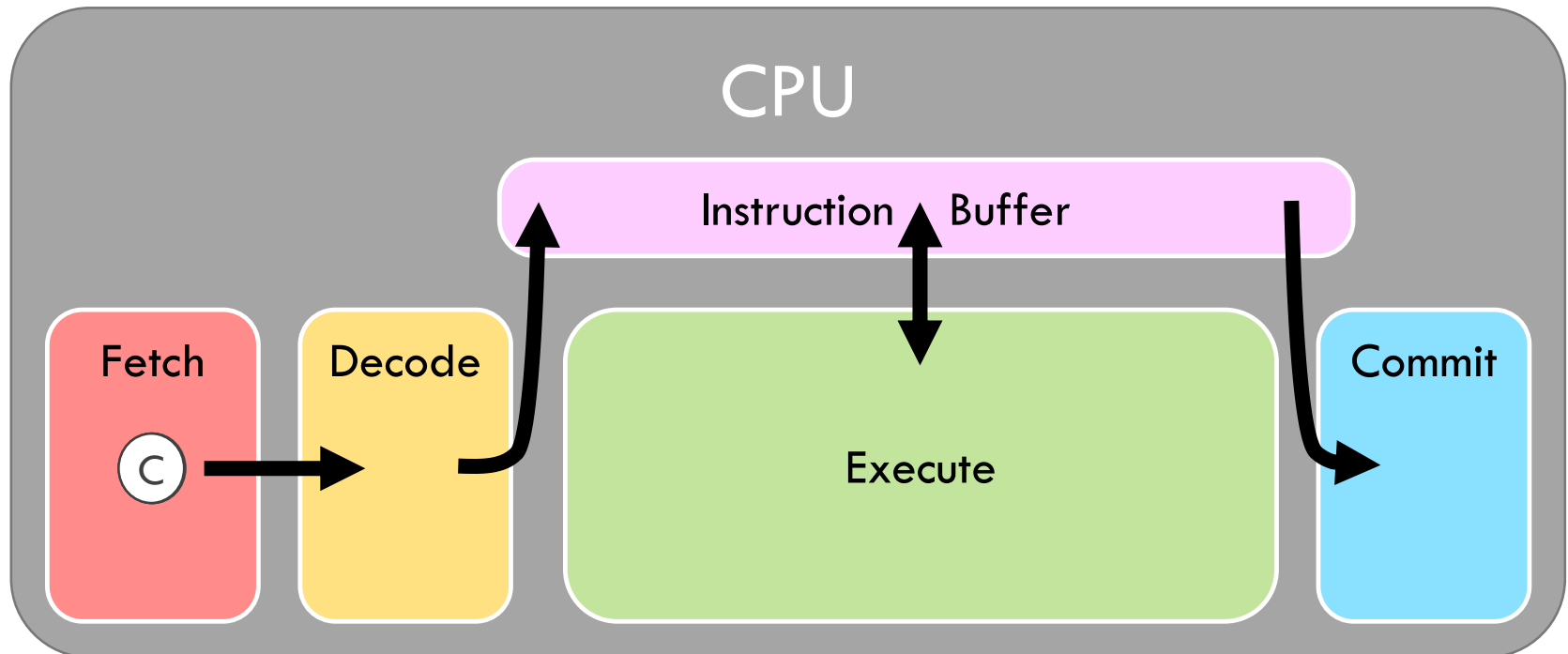
Out-of-order (OoO) execution uses dataflow to increase parallelism

- Idea: Execute programs in dataflow order, but give the *illusion* of sequential execution
- This is a “restricted dataflow” model
 - *Restricted* to instructions near those currently committing
 - (Pure dataflow processors also exist that expose dataflow to software)

High-level OoO microarchitecture



OoO is hidden behind in-order frontend & commit

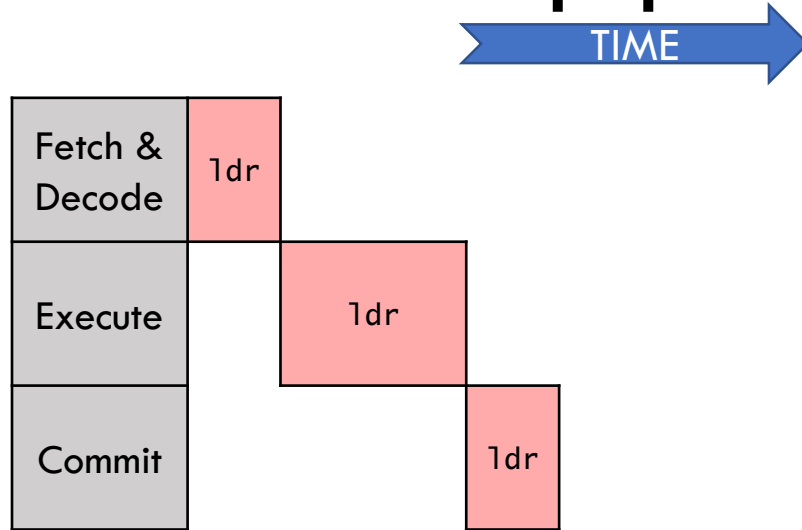


- Instructions only enter & leave instruction buffer in program order; all bets are off in between!

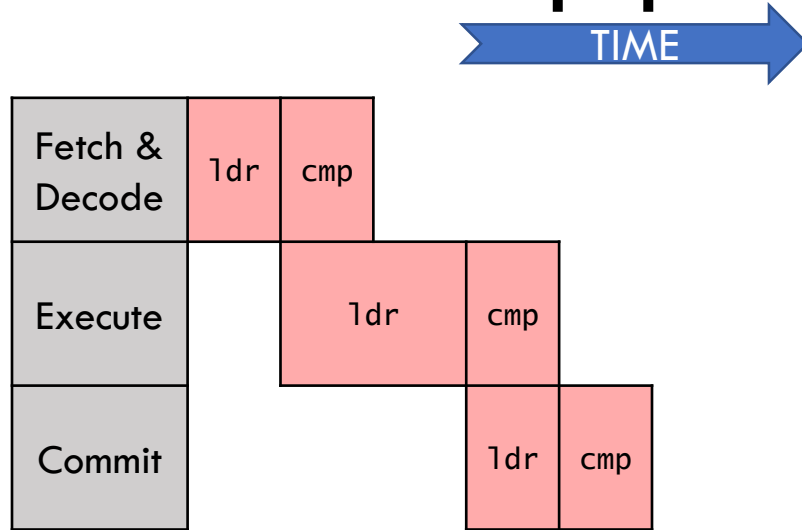
Example: OoO polynomial evaluation

- Q: Does OoO speedup execution? By how much?
- Q: What is the performance bottleneck?
- Assume perfect forwarding & branch prediction

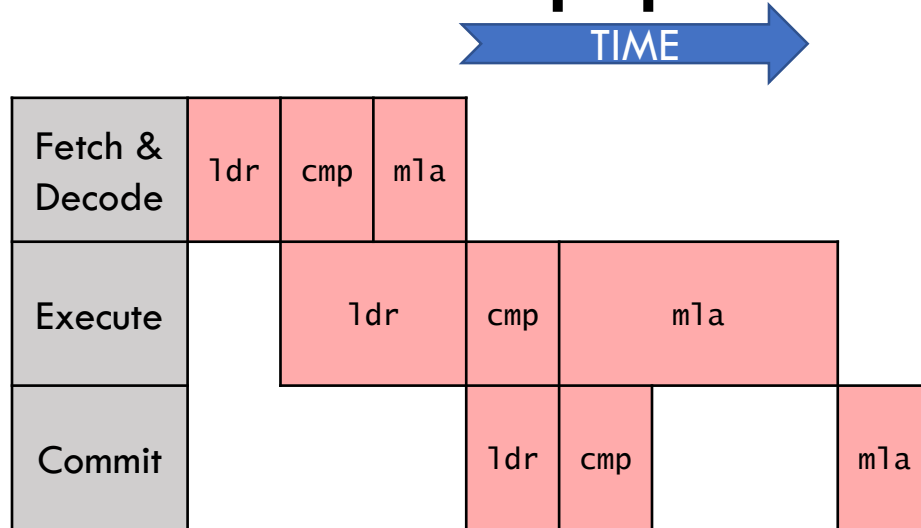
Example: OoO polynomial evaluation pipeline diagram



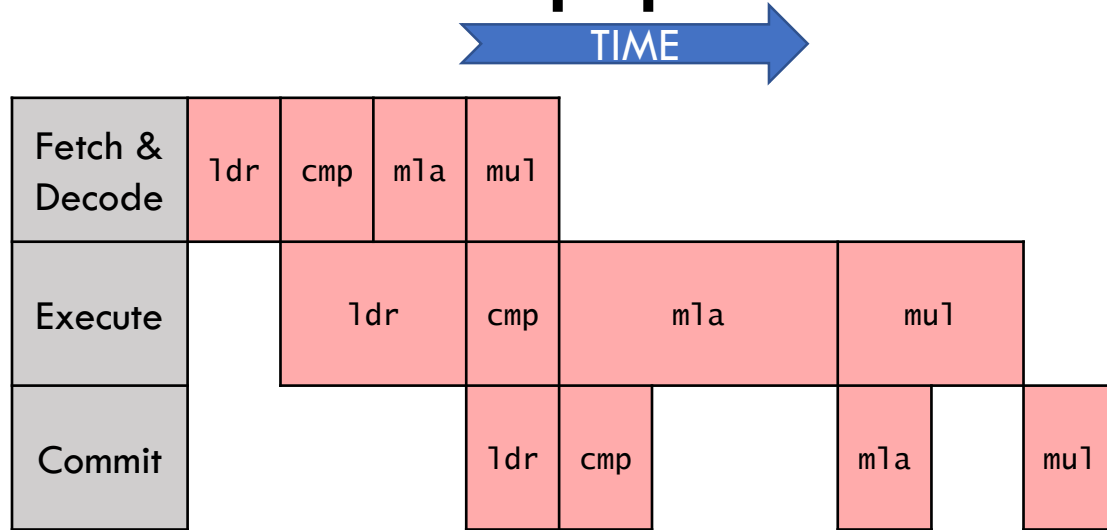
Example: OoO polynomial evaluation pipeline diagram



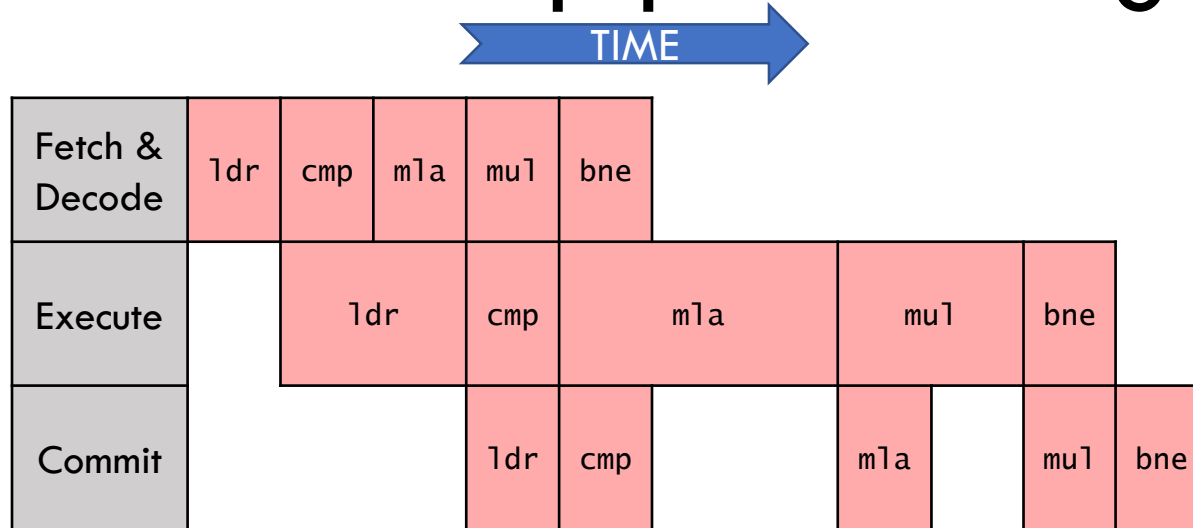
Example: OoO polynomial evaluation pipeline diagram



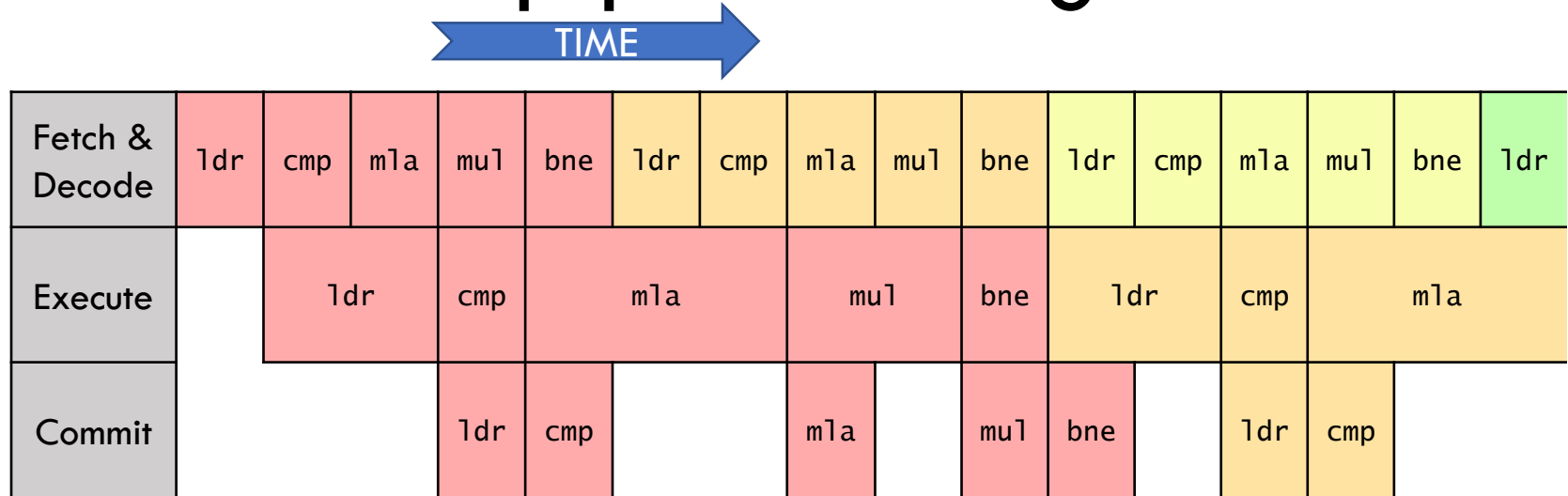
Example: OoO polynomial evaluation pipeline diagram



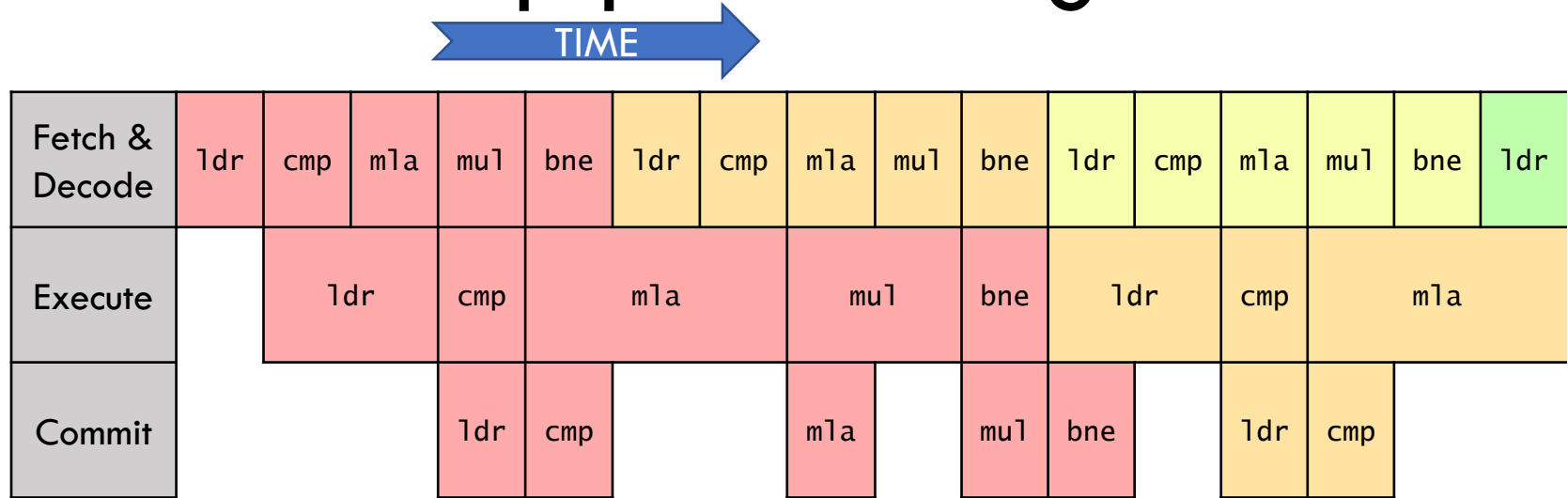
Example: OoO polynomial evaluation pipeline diagram



Example: OoO polynomial evaluation pipeline diagram



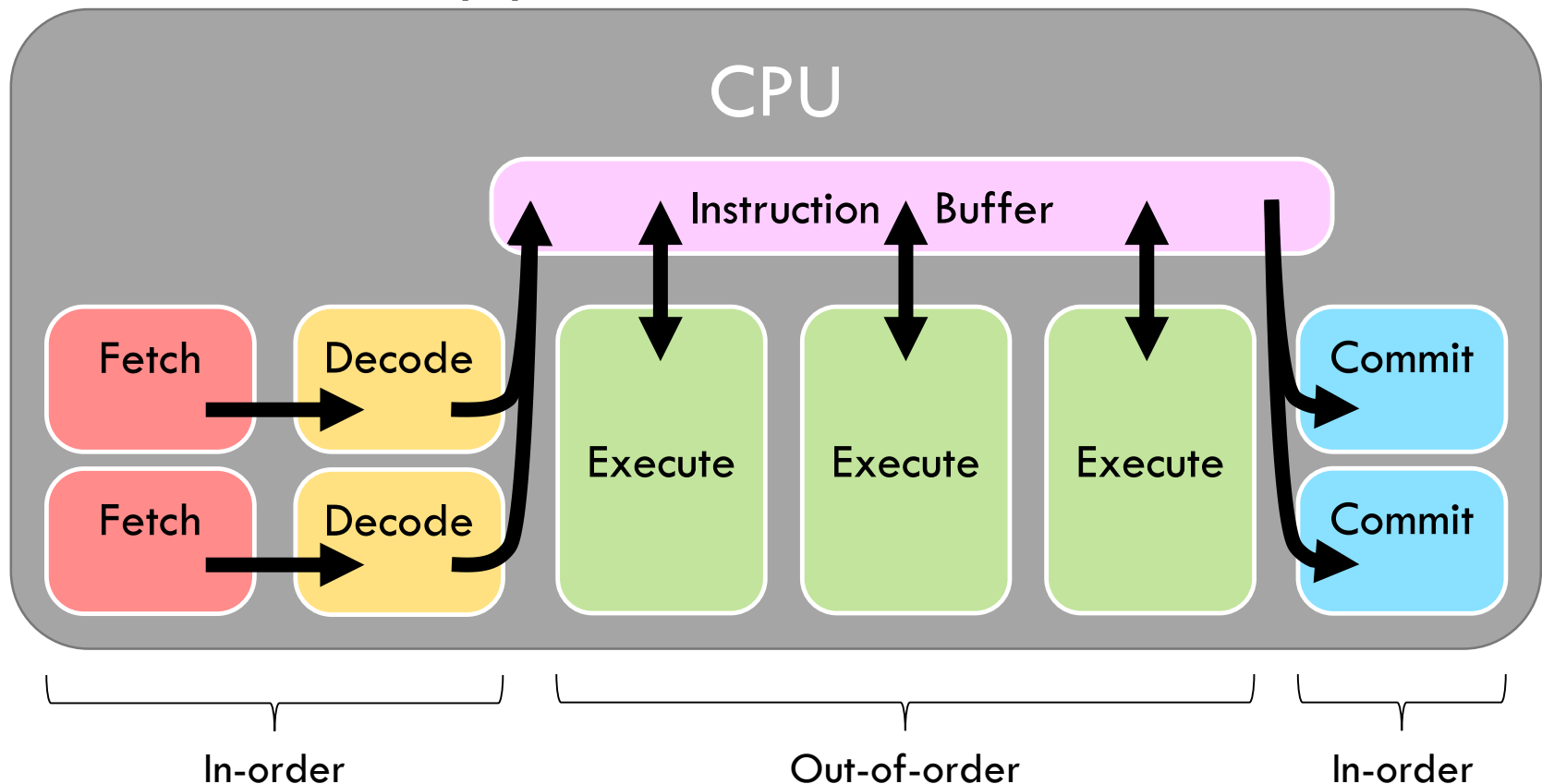
Example: OoO polynomial evaluation pipeline diagram



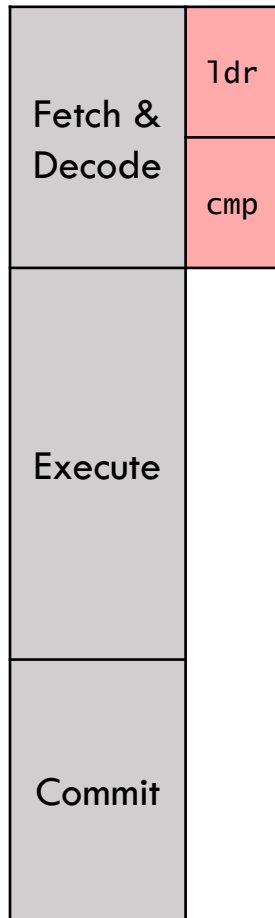
- Wait a minute... this isn't OoO... or even faster than a simple pipeline!
- Q: What went wrong?
- A: We're **throughput-limited**: can only issue 1 instrn

High-level **Superscalar** OoO microarchitecture

- Must increase *pipeline width* to increase $ILP > 1$



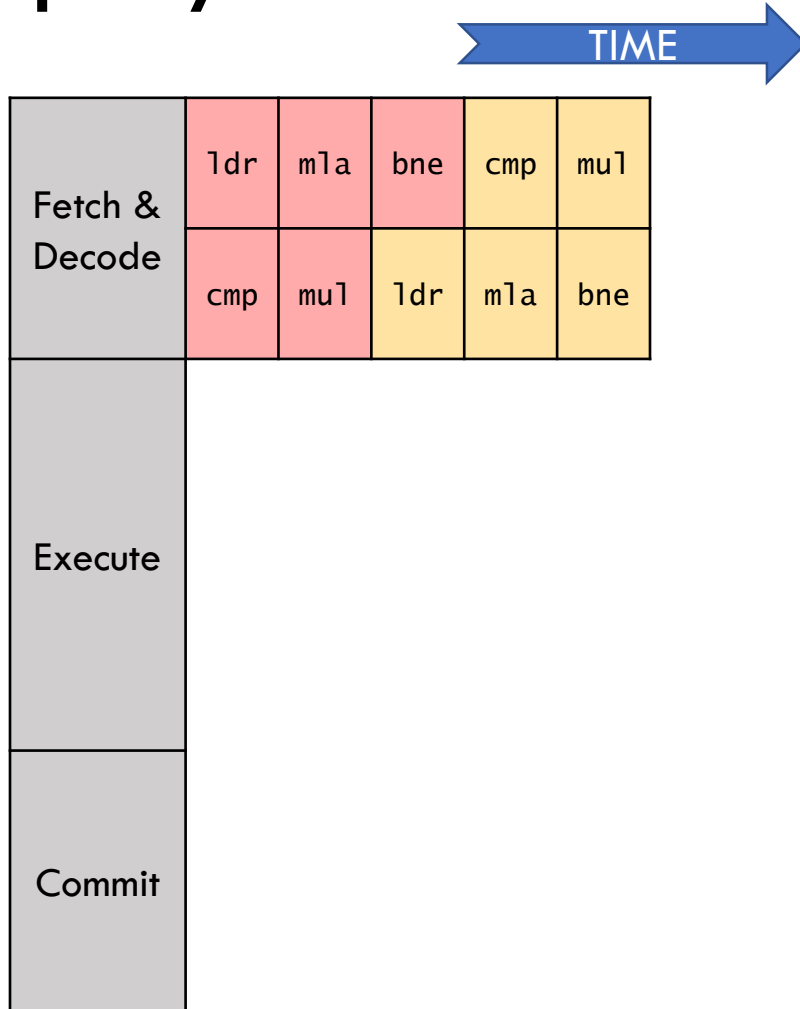
Example: Superscalar OoO polynomial evaluation



```

ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
ldr    r5, [r3], #4
cmp    r1, r3
mla    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
    
```

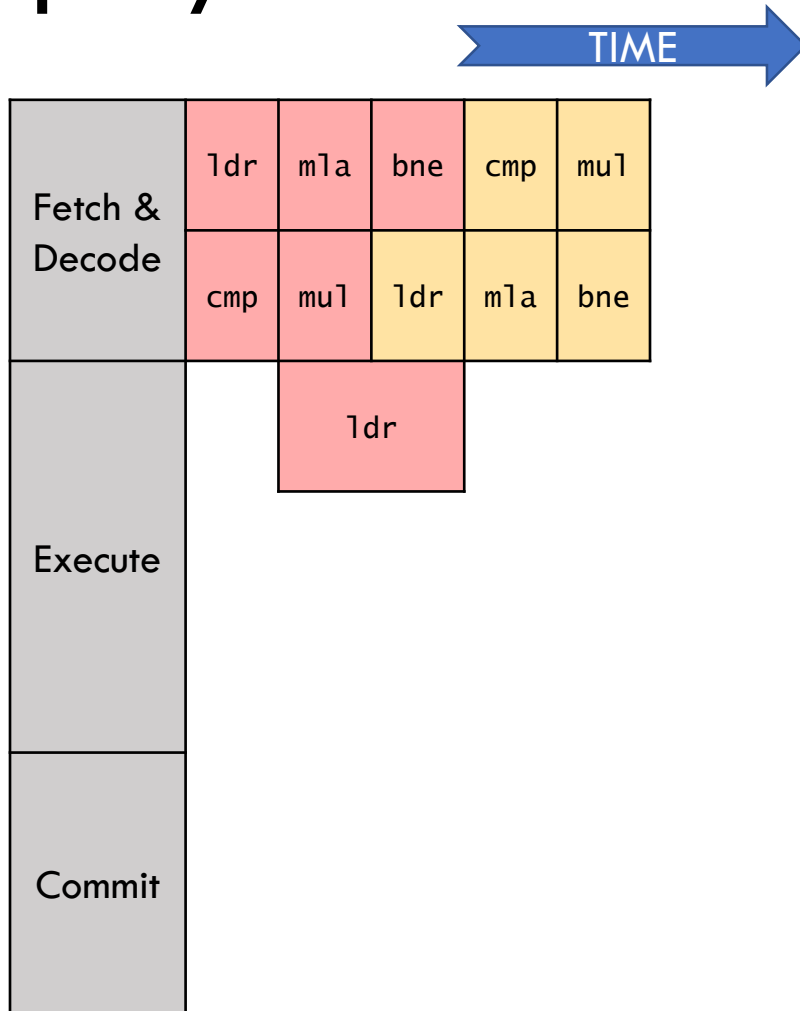
Example: Superscalar OoO polynomial evaluation



```

ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
    
```

Example: Superscalar OoO polynomial evaluation

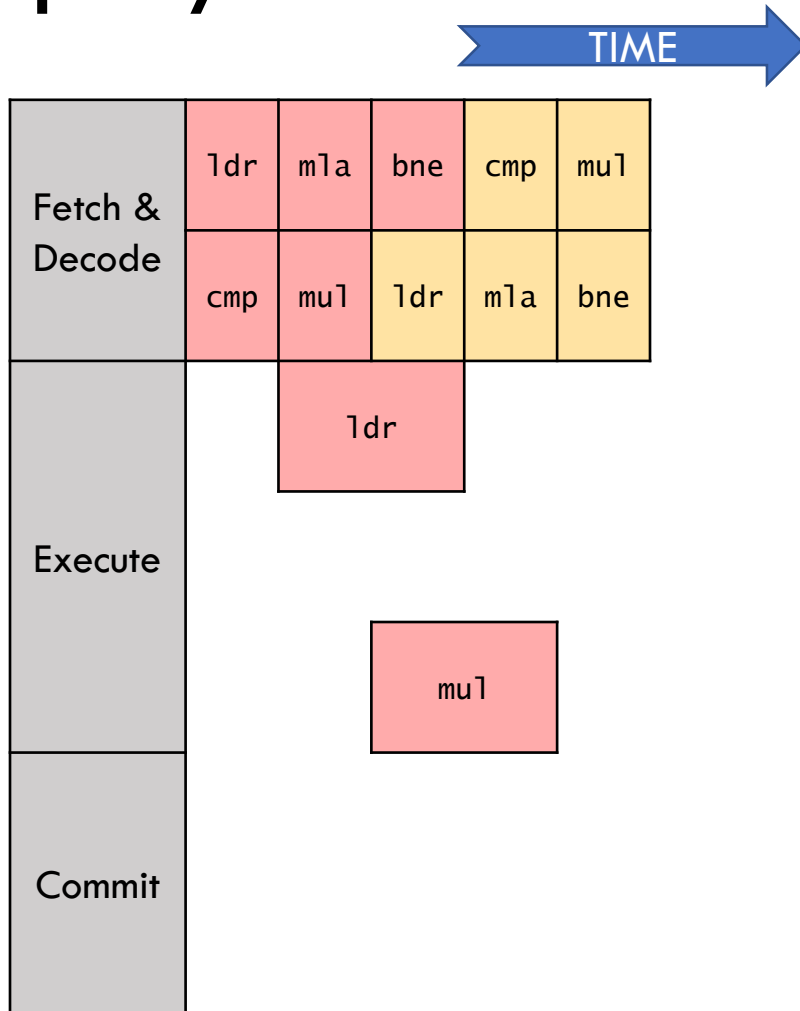


```

ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
    
```

Red arrows indicate data dependencies: from the first `ldr` to the first `cmp`, from the first `cmp` to the first `m1a`, and from the first `m1a` to the first `mul`. Another red arrow points from the first `mul` to the `.L3` label.

Example: Superscalar OoO polynomial evaluation

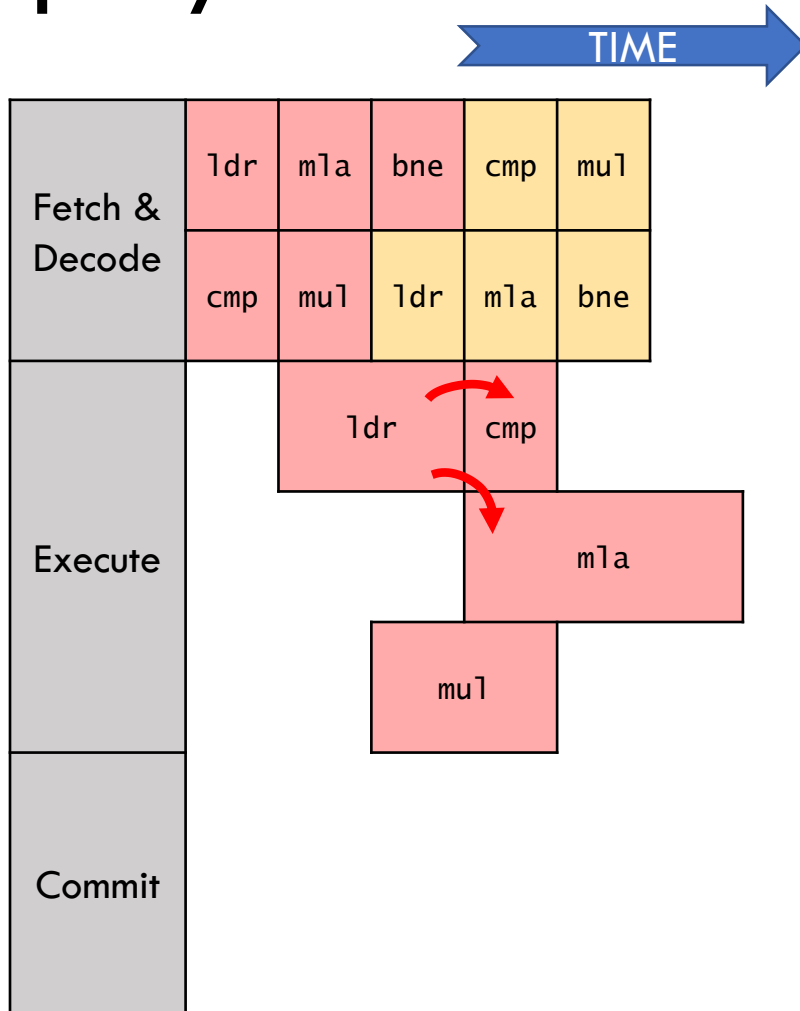


```

ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
    
```

Red arrows indicate data dependencies between instructions across cycles. For example, the `r5` register is updated by the first `ldr` and used by the first `cmp`, `m1a`, and `mul`. The `r3` register is updated by the first `ldr` and used by the first `cmp`. The `r4` register is updated by the first `mul` and used by the first `bne`. The `r0` register is updated by the first `m1a` and used by the first `mul`.

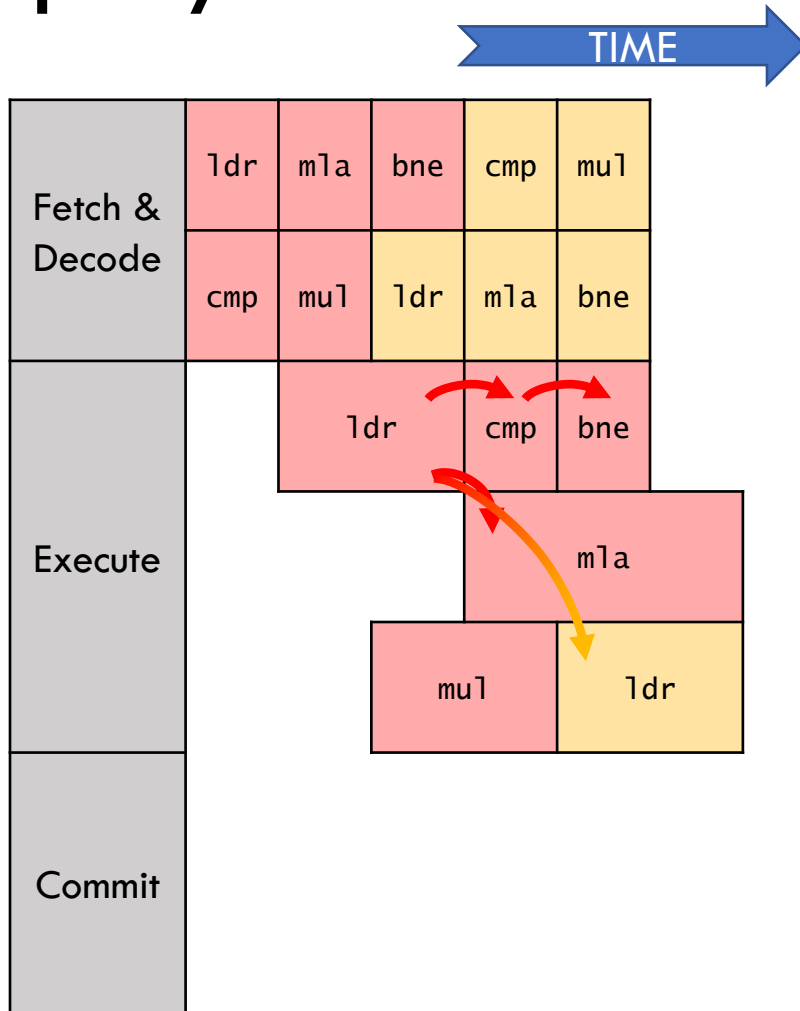
Example: Superscalar OoO polynomial evaluation



```

ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
    
```

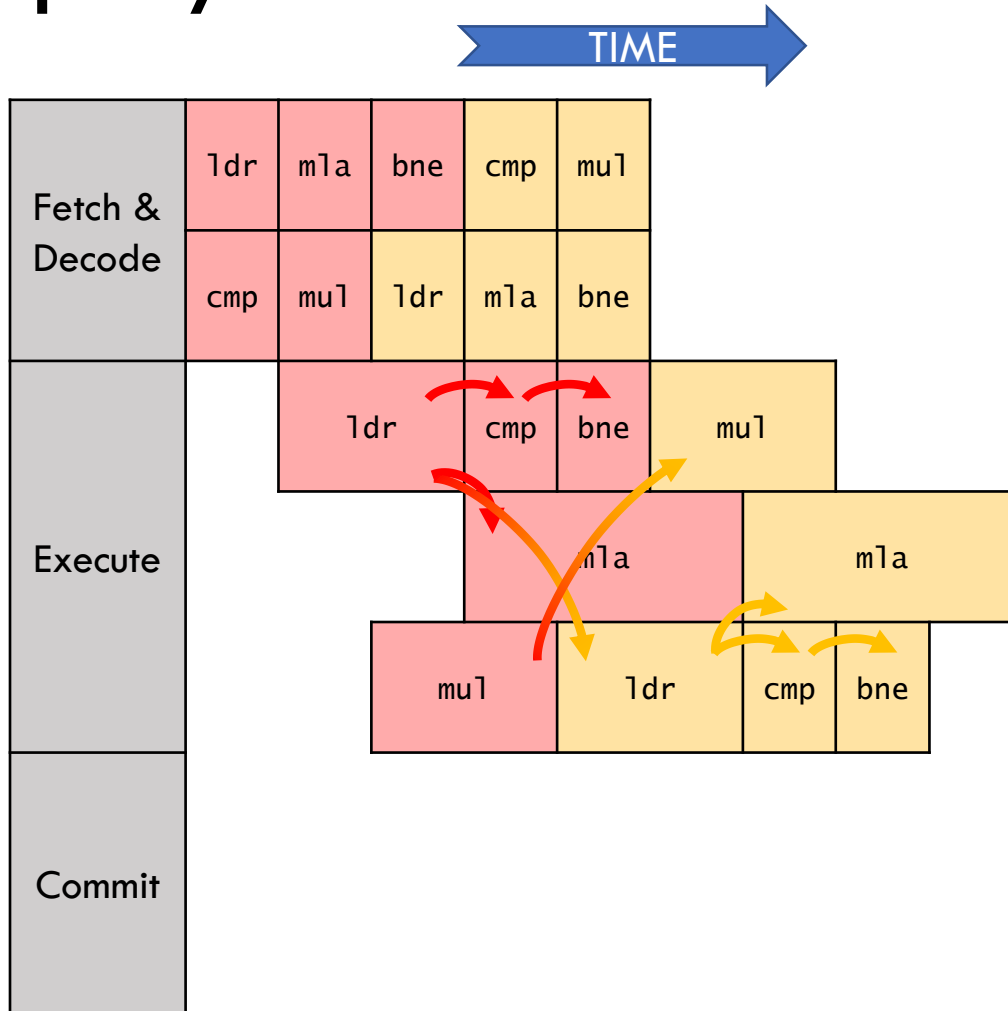
Example: Superscalar OoO polynomial evaluation



```

ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
    
```

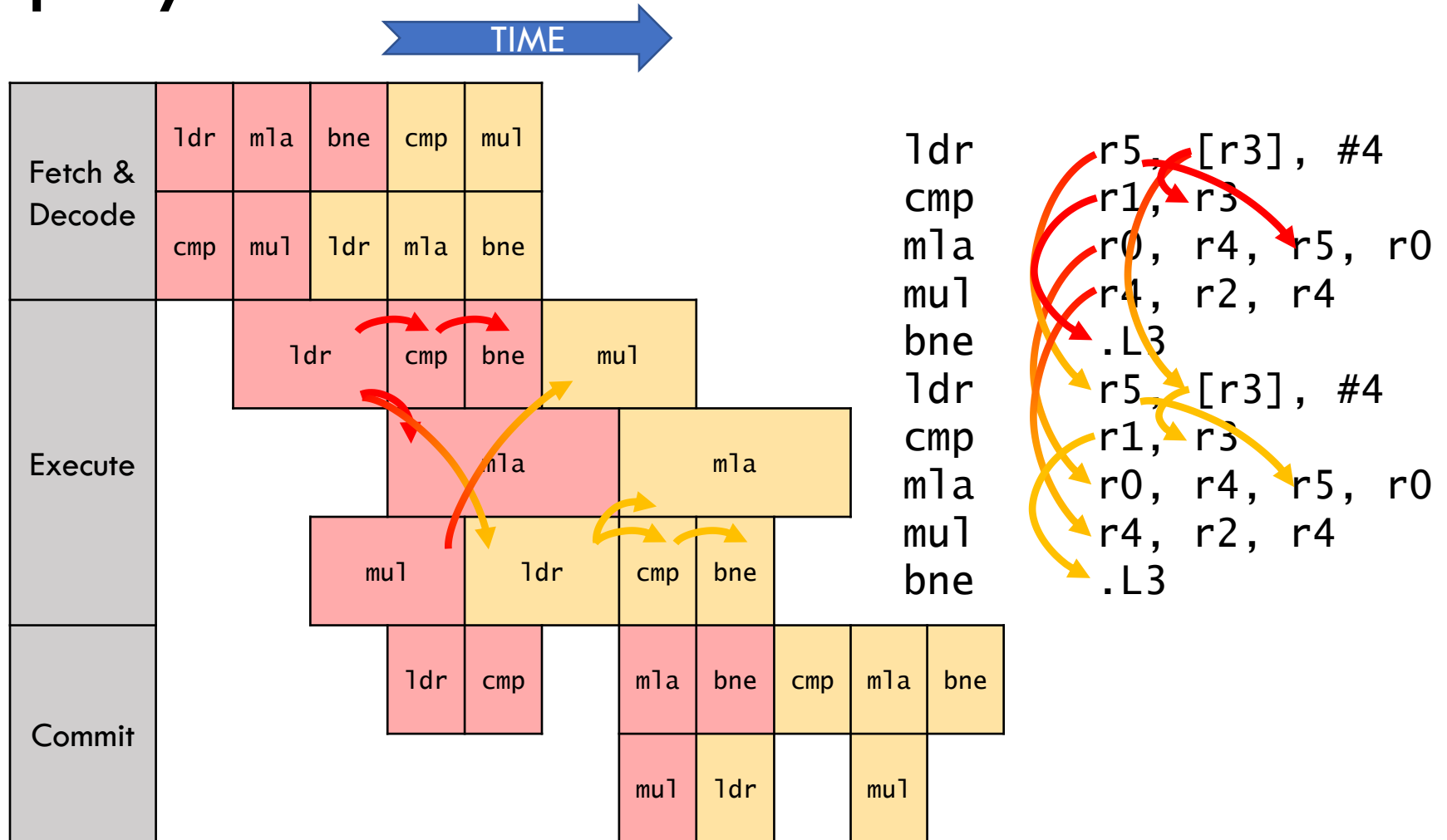
Example: Superscalar OoO polynomial evaluation



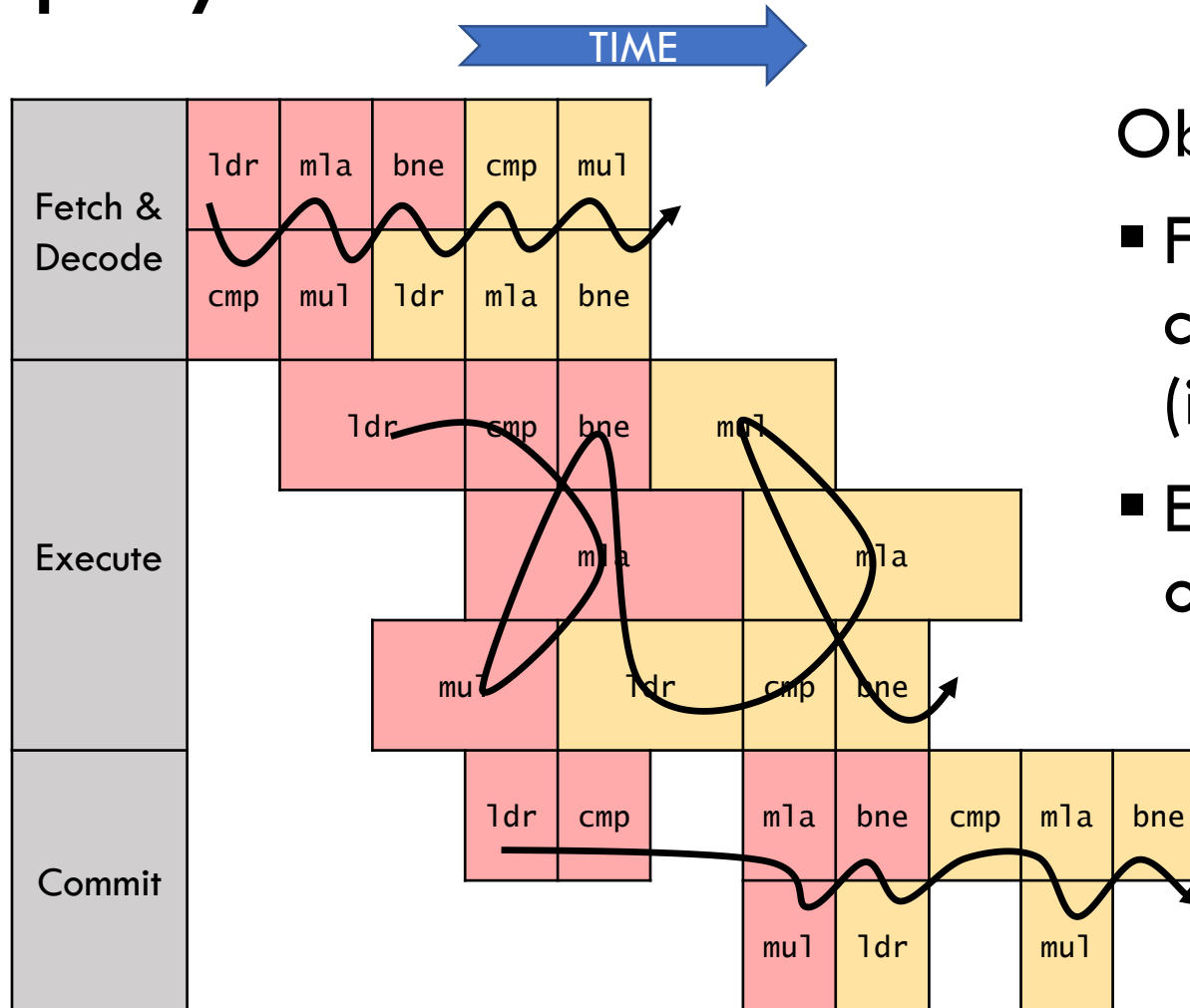
```

ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
ldr    r5, [r3], #4
cmp    r1, r3
m1a    r0, r4, r5, r0
mul    r4, r2, r4
bne    .L3
    
```

Example: Superscalar OoO polynomial evaluation



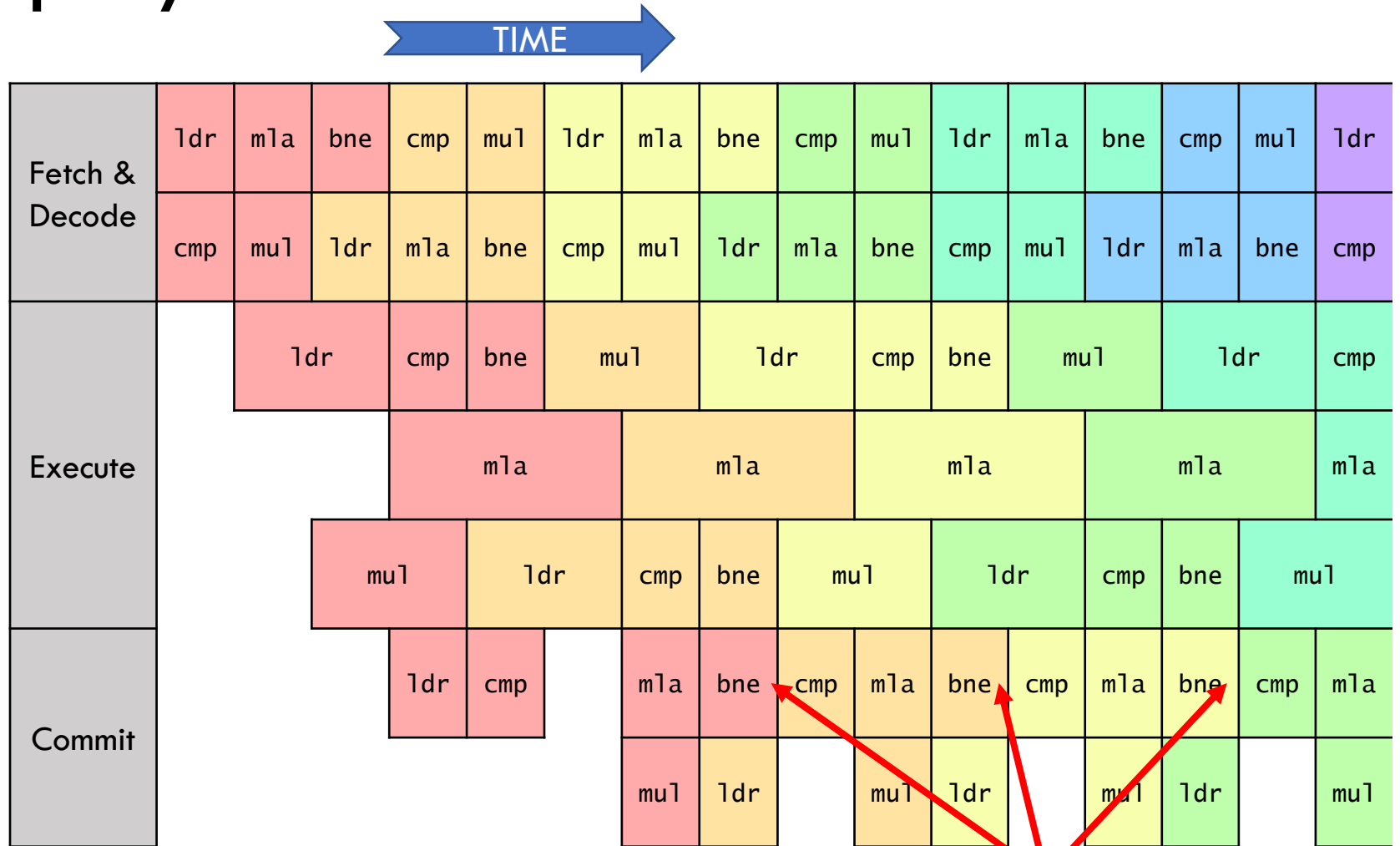
Example: Superscalar OoO polynomial evaluation



Observe:

- Front-end & commit in-order (i.e., left-to-right)
- Execute out-of-order

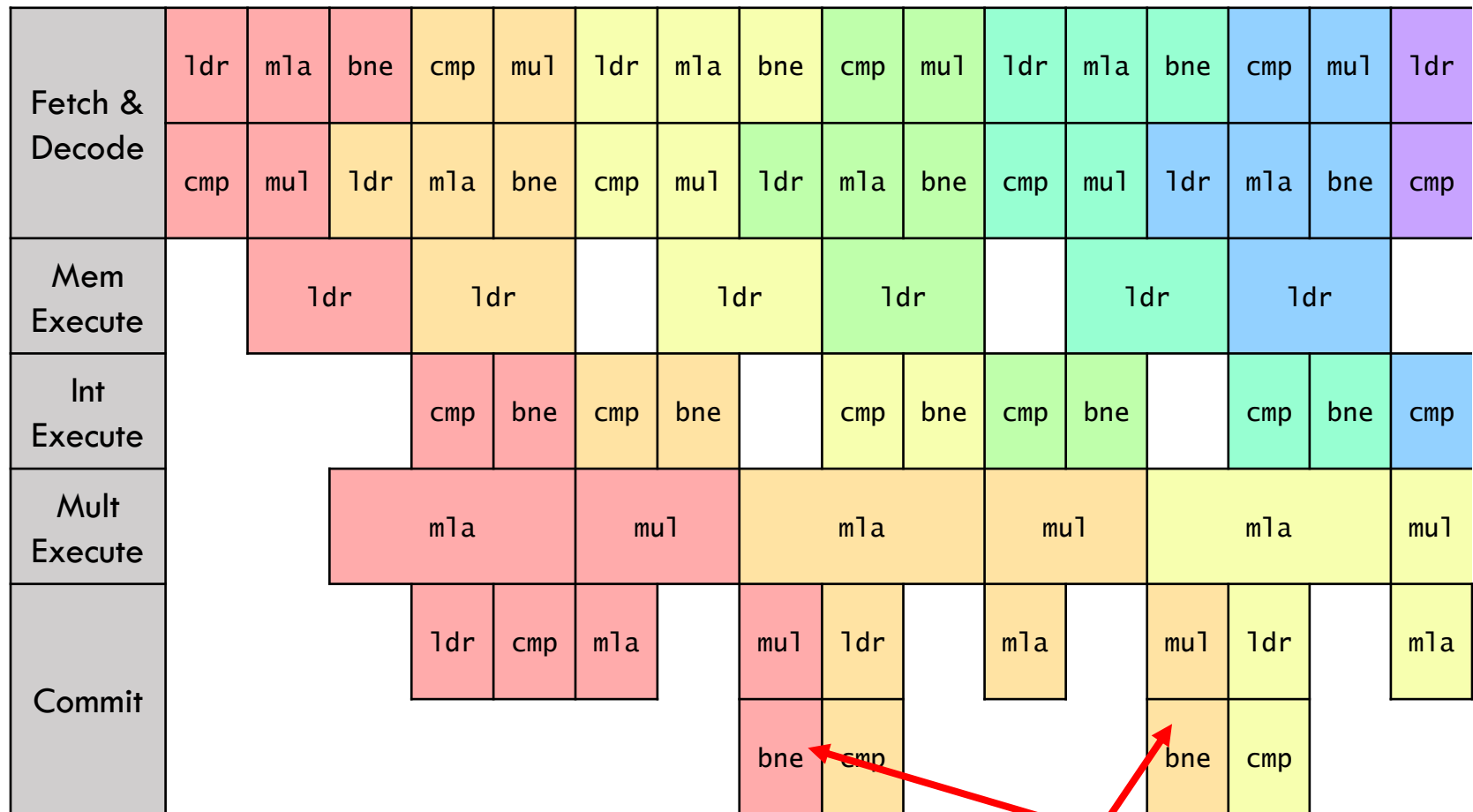
Example: Superscalar OoO polynomial evaluation



Structural hazards: Other throughput limitations

- Execution units are specialized
 - Floating-point (add/multiply)
 - Integer (add/multiply/compare)
 - Memory (load/store)
- Processor designers must choose which execution units to include and how many
- *Structural hazard*: Data is ready, but instr'n cannot issue because no hardware is available

Example: Structural hazards can severely limit performance



Superscalar scheduling is complex & hard to scale

- Q: When is it safe to issue two instructions?
- A: When they are independent
 - Must compare all pairs of input and output registers
- Scalability: $O(W^2)$ comparisons where W is issue width

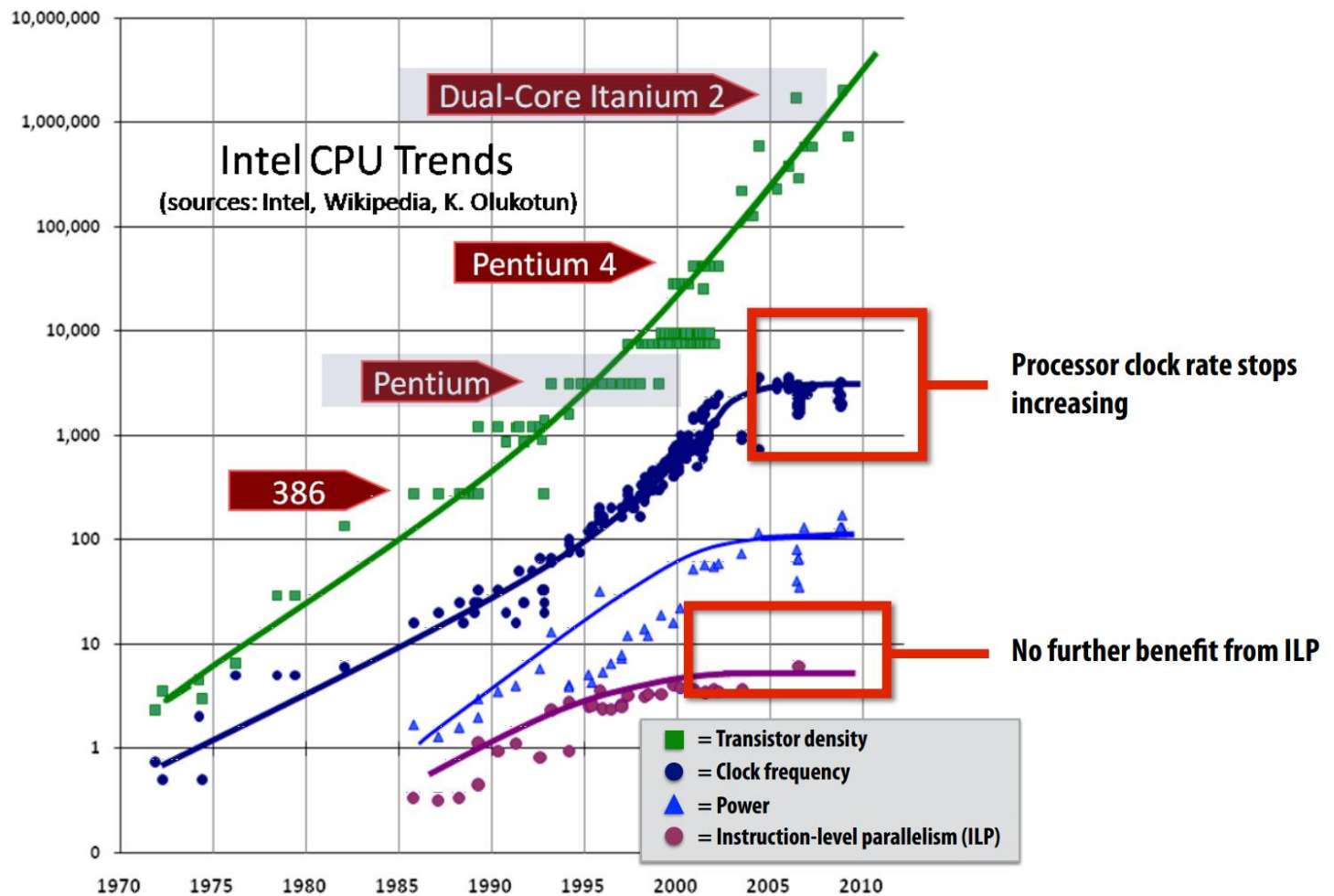
○○○ x86: Microcoding

- Each x86 instruction describes several operations
 - E.g., `add [esp+4], 5` means:
 1. Load `Mem[esp+4]`
 2. Add 5 to it
 3. Store result to `Mem[esp+4]`
- This is too much for (fast) hardware
- Instead, hardware decodes instr'ns into *micro-ops*
 - ➔ Rest of pipeline uses micro-ops

...But wait, there's more!

- Many issues we could not touch on
- How to eliminate *false dependences*
 - E.g., write-after-read / write-after-write
- How to track dependences through memory
 - E.g., store→load forwarding
- How to rollback mis-speculations

Recall from last time: ILP tapped out... why?



Limitations of ILP

- ILP works great! ...But is complex + hard to scale
- 4-wide superscalar $\times$ 20-stage pipeline = **80** instrns in flight
- High-performance OoO buffers hundreds of instructions
- Pipelines can only go so deep
 - Branch misprediction penalty grows
 - Frequency (GHz) limited by power
- Programs have limited ILP
 - Even with perfect scheduling, >8 -wide issue doesn't help
- Dynamic scheduling overheads are significant
- Out-of-order scheduling is expensive

Limitations of ILP → Multicore

- ILP works great! ...But is complex + hard to scale
- From hardware perspective, multicore is much more efficient, but...
- **Parallel software is hard!**
 - Industry resisted multicore for as long as possible
 - When multicore finally happened, CPU μ arch simplified → more cores
 - Many program(mer)s still struggle to use multicore effectively